Fundamentele programării 1.

Sorin Milutinovici
sorin.milutinovici@prof.utm.ro

Resurse utile

Prezentările sunt disponibile în secțiunea “Files” pe teams

Marea majoritate a exemplelor de cod sunt conectate cu editoare online de cod. Cel mai frecvent este OnlineGDB. Pentru cazul în care nu este disponibil, iată câteva alternative:

OnlineGDB beta online compiler and debugger for c/c++ | JDOODLE | paiza.io | ideone.com | IDE

http://tmpoll.greencore.ro/p/programare-introducere

Examenul va consta într-un test grilă de 20 de întrebări din programare C (în mare măsură) și câteva din conceptele generale

https://titumaiorescu.sharepoint.com/:f:/s/msteams_c40da4/Eoi8WExjn2JHnFXYgvSwqyUBASEtGlzjCbIBFJ8_OFL5tw?e=aocOh5

https://titumaiorescu.sharepoint.com/:f:/s/msteams_c40da4/El5BGFkXKKdGrT8swyeWS_kBingaoL6FNzSJ0fq06E8RmA?e=ZHQ1Br

https://titumaiorescu.sharepoint.com/:f:/s/msteams_a38777/Eu9qY4XjjJhLvlp39PBUwXMB35EfUNtmCbGDSDioTxWLmA?e=CEbmdr

https://titumaiorescu.sharepoint.com/:f:/s/msteams_54f290/EiGYNRAhrlJNkSz4S9Y0vmgBd4Upx-R7Q8hmMDqYT_HWIQ?e=PGr4hD

Structura cursului

- Mediul în care rulează programele: procesorul, numere binare, instrucțiuni
- Limbaje de programare - idee generală
- Paradigme de programare
- Algoritmi
- Limbajul de programare C
  - Prezentare generală
  - Variabile
  - Operatori
  - Instrucțiuni
  - Funcții
  - Tablouri de date
  - Pointeri
  - Șiruri de caractere
  - Fișiere
  - Structuri
  - Union, Enum

Vor exista 3 prelegeri (posibil prelungite în cea de-a patra ședință) plus două laboratoare.

Un program este o **suita de instrucțiuni** care sunt scrise de un **programator** pentru un **executant**

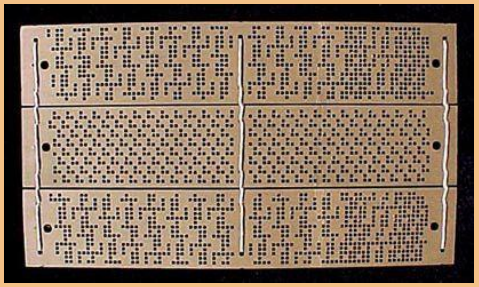
Ingrediente

- 1 1/2 căni maioneză
- 4 căței de usturoi, zdrobiți
- 1 linguriță sare
- 1 linguriță piper
- 2 lingurițe oregano
- 2 căni orez basmati
- 2 chili, feliați
- 1/3 cană apă
- 6 cartofi mai mici

Mod De Preparare

1. Amestecăm într-un caston maioneza cu sarea, piperul, usturoiul și oregano.
2. Puiul se acoperă cu toată această compoziție
3. Orezul se pune într-un vas de copt cu chili și se toarnă apoi apă peste.
4. Se adaugă cartofii și peste ei se pune puiul.
5. Se lasă la cuptor timp de 45 de minute la o temperatură de 180ºC.

1804 - Războiul de țesut Jacquard



https://www.youtube.com/watch?v=K6NgMNvK52A

compilator

00110101100101010100110011010010110011100101001100101011100111000111100101001100011100101001100110011010100110011010010110011100101001100011100101001100110011010100110011010010110011100101001100011100101001100110011010100110011010010110011

Procesorul

Procesorul reprezintă beneficiarul final al programatorului. Există, evident, o serie întreagă de dispozitive mult mai interesante și mai amuzante decât procesorul dar orice program, chiar dacă este gândit pentru o imprimantă 3D, ajunge să se execute pe un procesor în relație cu memoria.

Execuția unei instrucțiuni implică citirea ei din memorie, decodificarea acesteia și execuția ei. Fiecare astfel de acțiune se face la semnalul ceasului sistemului. De aceea, cu cât ceasul sistemului este mai rapid (mai multe bătăi pe secundă) programul se va executa mai repede.

Procesorul operează cu numere binare

0 0 0 1 1 1 0 0

$0 * 2^7 + 0 * 2^6 + 0 * 2^5 + 1 * 2^4 + 1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 0 * 2^0$

$0 + 0 + 0 + 16 + 8 + 4 + 0 + 0 = 28$

0 0 0 1 0 1 1 0

$0 * 2^7 + 0 * 2^6 + 0 * 2^5 + 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 1 * 2^1 + 0 * 2^0$

$0 + 0 + 0 + 16 + 0 + 4 + 2 + 0 = 22$

Adunarea a două numere

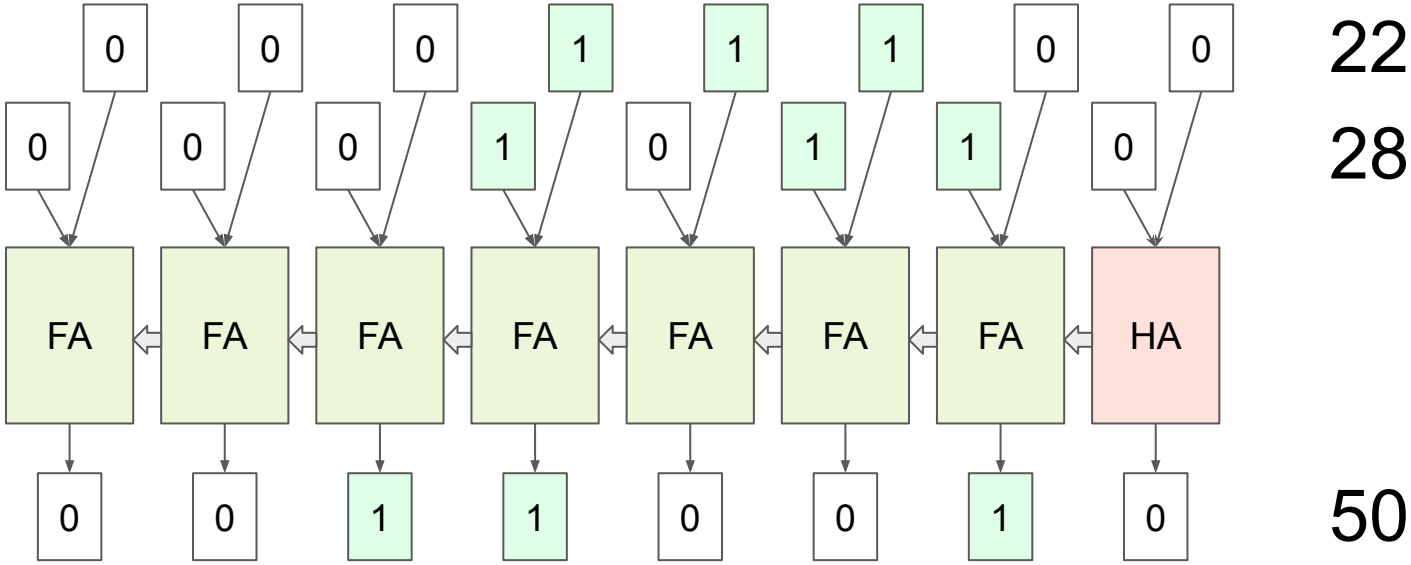
1		
2	2	+
2	8	
5	0	

			1	1	1			
0	0	0	1	1	1	0	0	+
0	0	0	1	0	1	1	0	
0	0	1	1	0	0	1	0	

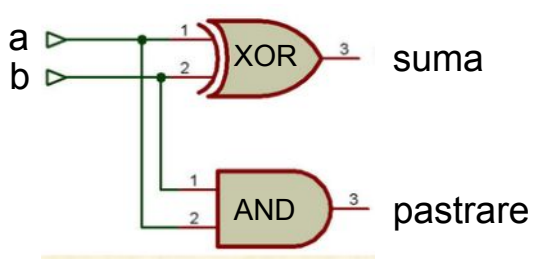
$1 * 2^5 + 1 * 2^4 + 1 * 2^1$

$32 + 16 + 2$

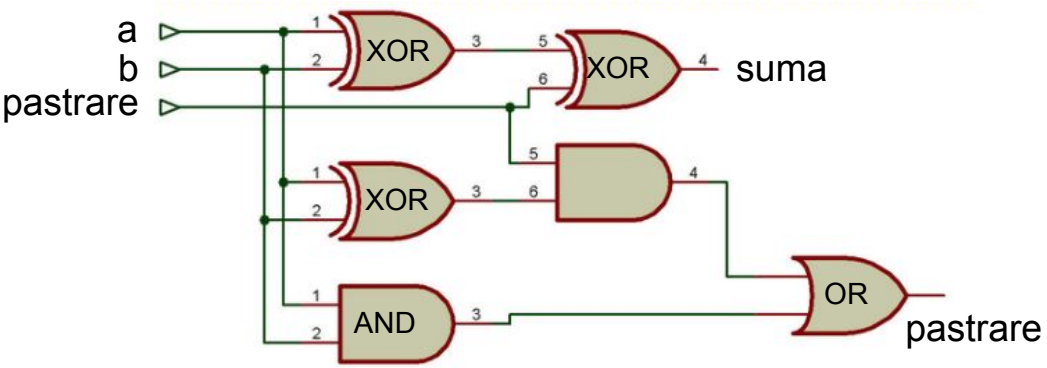
Implementare



HA = Half Adder (Sumator parțial)



FA = FULL Adder (Sumator integral)

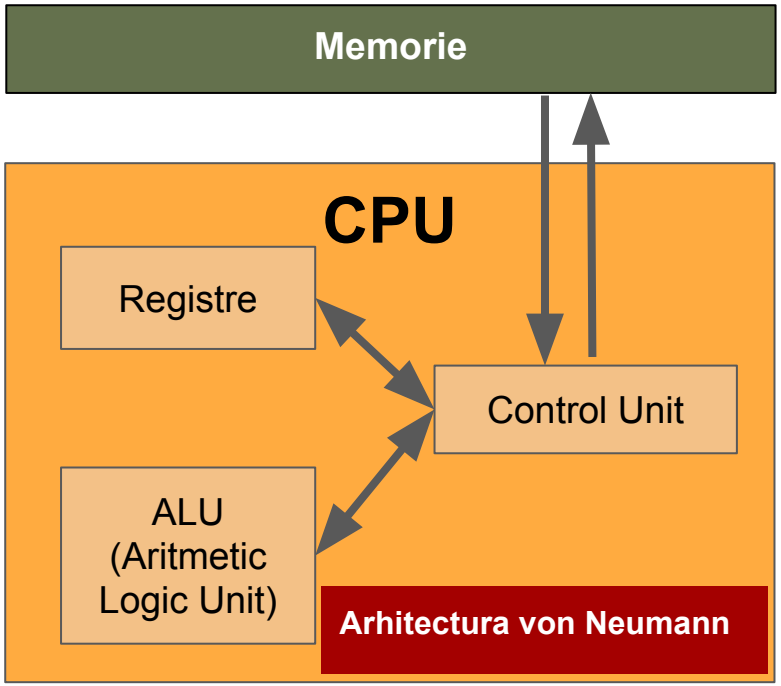


Implementare mecanica: https://www.youtube.com/watch?v=GcDshWmhF4A

Software = O listă de comenzi pentru procesor

Procesorul este în realitate beneficiarul nostru final, orice program, indiferent de modul în care este programat și de limbajul în care este programat va ajunge o simplă listă de comenzi pe care le execută procesorul

- Registre - memorii foarte rapide din care unitatea de control poate citi și în care poate scrie
- ALU - unitatea de calcule aritmetice
- Control Unit - unitatea care mută datele din memoria principală în regiștri și controlează ALU.



Un ciclu al procesorului mută datele pe traiectoria memorie -> registre -> ALU -> registre -> memorie.

Ordinele se numesc “instrucțiuni”

Program: R = 20 + 5

Programul este încărcat în memorie și procesat linie cu linie.

1. Copiază numărul stocat la adresa de memorie 10100 în registrul A (**LOAD**)
2. Adaugă numărul stocat la adresa de memorie 10101 celui stocat în registrul A (**ADD**)
3. Copiază conținutul registrului A la adresa de memorie 11111 (**STOR**)

Memorie

00000	10010100	LOAD 10100
00001	11010101	ADD 10101
00010	10111111	STOR 11111
00011	00000000	HALT
⋮		
10100	00010100	20
10101	00000101	5
10110	00000000	0
10111	00000000	0
11000	00000000	0

Procesorul va continua să execute toate liniile programului (instrucțiunile), una câte una, până când le termină.

Instruction Set Architecture

Totalitatea setului de instrucțiuni pe care le poate executa un anumit procesor se numește **Instruction Set Architecture**. Acest set de instrucțiuni depinde de tipul de procesor și **face parte din designul acestuia**. Setul este fix și este codificat hardware **în structura materială** a procesorului.

x86, MIPS, Itanium, Alpha, ARM, PowerPC, SPARC

https://en.wikipedia.org/wiki/Comparison_of_instruction_set_architectures

Probabil cel mai implementat set de instrucțiuni este x86, cu variantele lui pe 16, 32 și 64 de biți. Varietatea de implementări a determinat apariția unor grupuri de instrucțiuni suplimentare care nu fac parte din standardul de bază, numite extensii: **MMX** (extinderea registrelor), **3DNow!** (calcul vectorial), **SSE** (extensii la operațiile cu numere în virgulă mobilă), **PAE** (extensii la managementul memoriei),

Dimensiunea instrucțiunilor în seturile mari poate fi variabilă, în cazul X86 variază între 1 și 3 octeți. Pentru detalii, linkul următor.

https://www.intel.com/content/www/us/en/architecture-and-technology/64-ia-32-architectures-software-developer-vol-2a-manual.html

Programatorii, în special programatorii în limbajul C trebuie să aibă o imagine (oricât de generală) a modului în care funcționează procesorul. Implementările reale sunt extrem de complexe și constituie o specializare în sine. Pentru scopul acestui curs este suficient să folosim un model redus, un procesor virtual cu adrese pe 8 biti, cu un set de 8 instrucțiuni.

SimHYMN un program care simulează funcționarea unui procesor pe 8 biți

3 biti => 2^3 => 8 instructiuni http://www.cburch.com/socs/hymn/index.html

Setul de instrucțiuni SimHYMN

binar	nume	descriere
000	**HALT**	Oprește execuția programului
001	**JUMP**	Execută un salt la instrucțiunea de la adresa specificată
010	**JZER**	Jump on zero
011	**JPOS**	Jump on positive
100	**LOAD**	Copiază instrucțiunea de la adresa dată în acumulator
101	**STOR**	Copiază valoarea din acumulator în locația de memorie cu adresa dată
110	**ADD**	Adaugă valoarea din acumulator la valoarea stocată în adresa de memorie dată și copiază rezultatul în AC
111	**SUB**	Scade valoarea de la adresa de memorie dată din valoarea din acumulator

Adrese speciale de memorie		
	11110	Input de la consolă (Memory Mapped IO)
	11111	Output către consolă (Memory Mapped IO)

Program: Compararea a două numere

```
10011110
10101011
10011110
11101011
01001000
10001100
10111111
00000000
10001101
10111111
00000000
00000000
11111111
00000001
```

Prima abstractizare a limbajului mașină - limbajul de asamblare

Adunare

adresă	valoare stocată la adresă	Instrucțiuni	Limbaj de asamblare	
00000	10011110	LOAD11110	READ	#Citește o valoare de la IO și o plasează în acumulator
00001	10100110	STOR00110	STOR Va	#Stochează valoarea citită în variabila Va
00010	10011110	LOAD11110	READ	#Citește o valoare de la IO și o plasează în acumulator
00011	11000110	ADD 00110	ADD Va	#Adaugă valoarea variabilei Va în acumulator
00100	10111111	STOR11111	WRITE	#Scrie valoarea din acumulator la IO
00101	00000000	HALT	HALT	#Opre□te programul
00110	00000000	00000000	Va:0	#Declară valoarea 0 pentru variabila Va

Scădere

adresă	valoare stocată la adresă	Instrucțiuni	Limbaj de asamblare	
00000	10011110	LOAD11110	READ	#Citește o valoare de la IO și o plasează în acumulator
00001	10100110	STOR00110	STOR Va	#Stochează valoarea citită în variabila Va
00010	10011110	LOAD11110	READ	#Citește o valoare de la IO și o plasează în acumulator
00011	11100110	SUB 00110	SUB Va	#Scade valoarea variabilei Va din acumulator
00100	10111111	STOR11111	WRITE	#Scrie valoarea din acumulator la IO
00101	00000000	HALT	HALT	#Opre□te programul
00110	00000000	00000000	Va:0	#Declară valoarea 0 pentru variabila Va

Inmultire

```
# SimHYMN assembler file
#
# Cere succesiv doua numere la consola si afiseaza produsul lor
# se incarca in programul simhymn
# http://www.cburch.com/socs/hymn/index.html
#

LOAD prod                #Reseteaza valoarea variabilei prod
SUB prod                   #la 0 pentru a permite mai multe
STOR prod                #rulari succesive ale programului
READ                          #Citeste o valoare de la IO
STOR Va                       #Stocheaza valoarea citita in variabila Va
READ                          #Citeste o valoare de la IO
STOR Vb                       #Stocheaza valoarea citita in variabila Vb
STOR decr                     #Stocheaza valoarea citita in variabila de decrementare
LOOP:                         #Porneste o bucla
    LOAD prod                 #Incarca variabila prod in acumulator
    ADD Va                     #Adauga variabila Va
    STOR prod                 #Stocheaza noua valoare a prod
    LOAD decr                 #Incarca variabila de decrementare in acumulator
    SUB one                    #Scade valoarea variabilei one (1) din acumulator
    STOR decr                 #Copiaza in memorie descr
JPOS LOOP                     #Sare inapoi la LOOP daca val din acumulator e pozitiva
LOAD prod                     #Incarca Va in acumulator
WRITE                         #Scrie valoarea din acumulator la IO
HALT                          #Opreste programul
Va:0                          #declara valoarea 0 pentru variabila Va
Vb:0                          #declara valoarea 0 pentru variabila Vb
decr:0                        #declara valoarea 0 pentru variabila decr
one:1                         #declara valoarea 1 pentru variabila one
prod:0                        #declara valoarea 0 pentru variabila prod
```

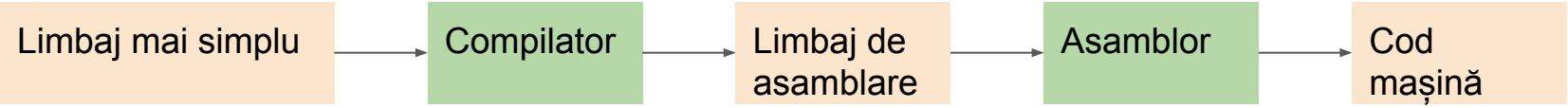
Dezavantajele programării în cod mașină

- Codul binar este foarte dificil de citit
- Comenzile disponibile sunt puține și primitive
- Programele sunt foarte lungi și foarte greu de depanat
- Limbajul mașină nu oferă funcții matematice, ele trebuie scrise de fiecare dată
- Locațiile de memorie trebuie manipulate direct făcând imposibil lucrul cu cantități mari de memorie

Dezavantajele programării în limbaj de asamblare

- Limbajul de asamblare este în continuare foarte dificil de citit
- Comenzile disponibile nu sunt mult mai multe decât cele în cod mașină
- Programele sunt foarte lungi și foarte greu de depanat
- Programele scrise în limbaj de asamblare sunt dependente de sistemul de operare și de arhitectură (asamblorul este un program ca oricare altul și rulează doar pe un anumit sistem de operare.

Se poate construi un limbaj mai simplu împreună cu un “traducător” care să genereze automat limbaj de asamblare?



Limbaje de programare

Cum ar trebui să arate?

1. Variabilele trebuiesc inițializate
2. Compilatorul trebuie să știe ce vom stoca în ele (numere întregi, șiruri)

```
altx = 10;
```

```
int altx = 10;
```

3. Putem concepe un sistem de teste la care programul să sară la diverse adrese de memorie, executând diferite secvențe de cod.

```
if( altx < 20 ) {
   /* executa bloc a */
 } else {
   /* executa bloc b */
 }
```

4. Putem concepe un sistem de instrucțiuni care să permită execuția repetată a unei anumite secvențe de instrucțiuni

```
for( a = 10; a < 20; a = a + 1 ){
    printf("value of a: %d\n", a);
 }
```

5. Putem concepe un sistem prin care să înregistrăm secvențe de instrucțiuni la anumite adrese de memorie, dincolo de instrucțiunea HALT, care să fie executate dacă sunt apelate din interiorul codului principal

```
int max(int num1, int num2) {

   /* local variable declaration */
   int result;
   if (num1 > num2)
      result = num1;
   else
      result = num2;
   return result;
}
```

Limbaje de nivel înalt

- Limbaj mult mai apropiat de limbajul natural
- Operații matematice predefinite care pot fi apelate prin operatori simpli (>, <, *, /, ^ etc.)
- Operații logice predefinite (AND, OR etc.)
- Instrucțiuni care definesc blocuri de cod repetitive, condiționale
- Suport automat pentru operații de Input/Output comune.
- Diferite posibilități adăugate limbajului în funcție de scopul pentru care a fost creat.

Ethnologue - languages of the world (https://www.ethnologue.com/guides/how-many-languages) înregistrază 7,150 limbi vorbite pe pământ.

Online Historical Encyclopaedia of Programming Languages (http://hopl.info/) înregistrază 8945 limbaje de programare.

De ce sunt atât de multe limbaje de programare?

Absolut toate ajung aici:

```
10011110
10101011
10011110
11101011
01001000
10001100
10111111
00000000
10001101
10111111
00000000
00000000
11111111
00000001
```

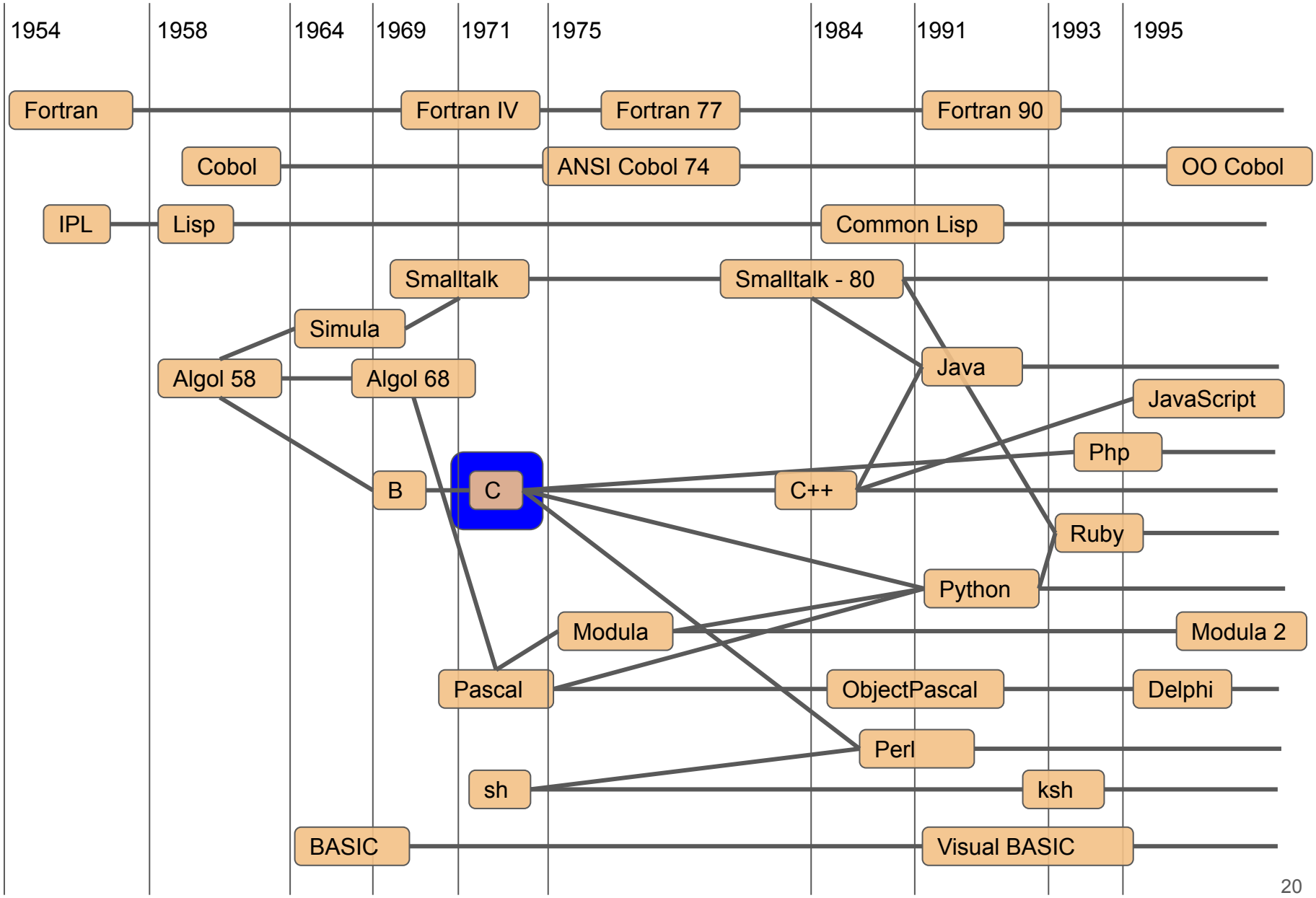
Fiecare program exprima una sau mai multe idei.

Crearea unui program nu are ca scop doar automatizarea unui procesor ci, de multe ori mult mai important, exprimarea unor idei și concepte.

Limbajul este limitat iar exprimarea conceptelor este dependentă de el. Obiceiul limbii germane de a înlănțui cuvinte permite exprimarea eficientă unor concepte complicate (cum este celebrul Schadenfreude care este compus din Schaden (stricăciune, defecțiune) și freude (bucurie). Alte limbi au dificultăți în exprimarea aceluiași concept.

Toate limbajele de programare, toate paradigmele în care acestea se găsesc, toate sintaxele inventate în acest domeniu, toate acestea sunt pentru oameni, nu pentru computere. Ca atare, nu e neobișnuit să găsim în ele o componentă psihologică și socială semnificativă.

Evoluția limbajelor de programare



Paradigme de programare

Paradigmele sunt modele de rezolvare a unor probleme. Având în vedere că dezvoltarea unui program nu înseamnă doar generarea unei succesiuni de 1 și 0, ci exprimarea unor idei umane, e oarecum firesc ca de-a lungul timpului să se fi dezvoltat mai multe tipuri distincte de abordări pentru rezolvarea aceleiași probleme

Aceeași problemă se poate rezolva în toate stilurile posibile. Anumite paradigme sunt mai potrivite pentru anumite clase de probleme, altele pentru alte clase de probleme. Î

Conceptul a fost definit de Robert Floyd și a fost prezentat într-o comunicare ținută cu ocazia primirii premiului Turing. Comunicarea poate fi citită aici

Paradigme de programare

imperative

Programele sunt compuse din instrucțiuni care spun computerului ce are de făcut și prin ce metode să ajungă la scopurile definite. Operația principală este atribuirea unor valori variabilelor definite.

- procedurale: Programele sunt împărțite în mai multe proceduri care se pot apela una pe cealaltă.
- Orientate pe obiecte: Programele sunt definite ca o sumă de componente care transmit mesaje între ele. Componentele pot fi extinse cu noi caracteristici dând naștere unor noi obiecte în timpul rulării.
- Orientate pe eveniment: Programul este o lista a tuturor evenimentelor posibile, cu acțiunile corespunzătoare fiecăruia.

declarative

Programele sunt compuse din instrucțiuni care spun computerului ce are de făcut fără să menționeze modul - Operația principală este descrierea unei stări.

- DSL (domain specific language): Colecții de funcții special proiectate pentru un anumit domeniu de interes cu utilizare restrânsă
- funcționale: Programele sunt definite ca o colecție de funcții matematice.
- logice: Programele sunt definite ca o succesiune de axiome urmate de relații logice între date. Utilizate în special în lucrul cu colecții mari de informație cu relații cunoscute.

Programarea și procesoarele

Nu toate programele ajung să fie executate de câtre un procesor. O mică parte din ele pot fi utilizate în descrierea și implementarea unor sisteme.

Exemple:

ARM, Bicep - limbaj declarativ pentru descrierea configurării infrastructurilor (Infrastructure as Code) în sistemul Azure cloud.

AWS CloudFormation - Infrastructure as code pentru Amazon cloud

Terraform - Infrastructure as code pentru mai multe sisteme de cloud.

Verilog/VHDL- topologia circuitelor FPGA

UML - Limbaj care permite construcția unor diagrame specifice mergând până la generatoare de cod.

Programare reactivă

O paradigmă în care un program este privit ca o rețea arborescentă de noduri care reacționează la prezența datelor în funcție de modul în care au fost configurate.

Nodurile pot conține operații de tip map, filter, join, window, scan, merge, split, etc.

Conexiunile dintre noduri pot fi de mai multe moduri: unicast, broadcast , uniune, filtrare, buffering, etc.

Exemplul clasic îl reprezintă o foaie de calcul tabelar care are o serie de formule. Formulele "reacționează de câte ori se modifică datele de intrare din celula desemnată ca input.

Un alt exemplu - sistemul de validare a unui formular web - sistem care reacționează la datele de intrare generând mesaje de eroare sau permițând înregistrarea datelor în sisteme de stocare.

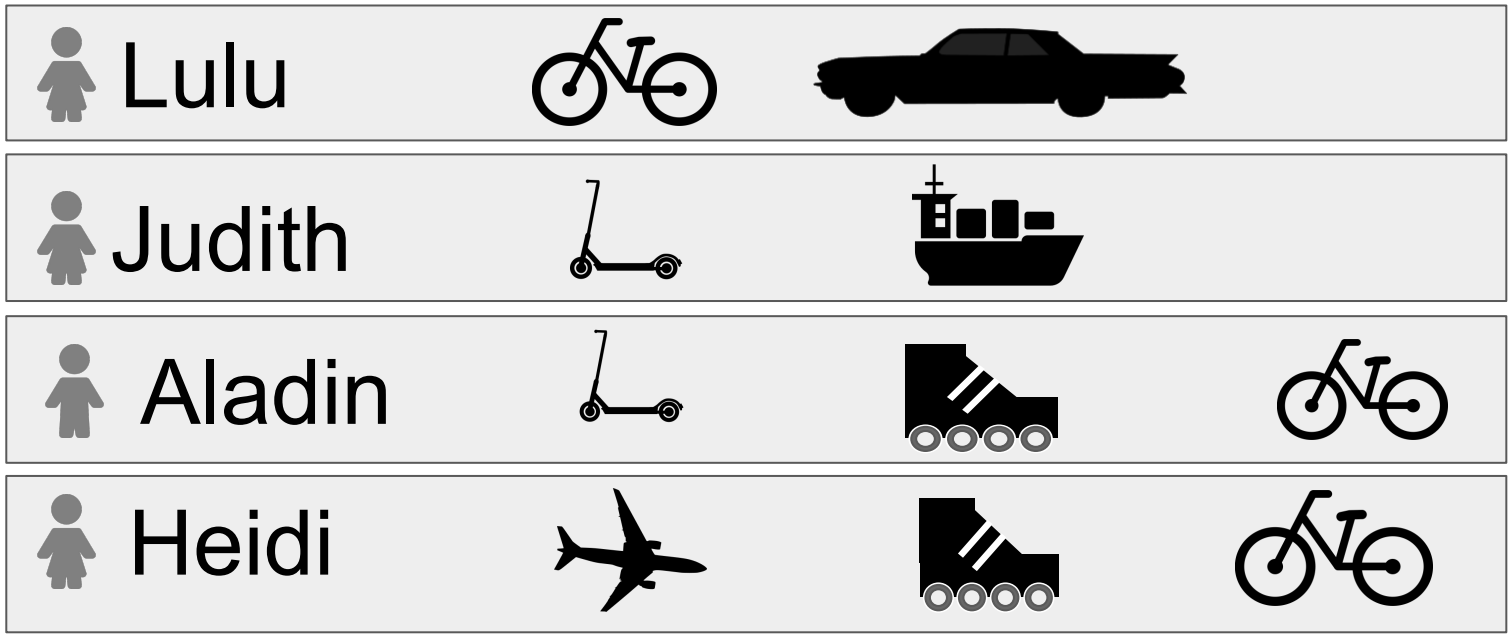
Nodurile unui sistem reactiv pot fi programare imperativ, sistemul propriu-zis poate fi programat imperativ sau declarativ, în cazul în care există limbaje DSL

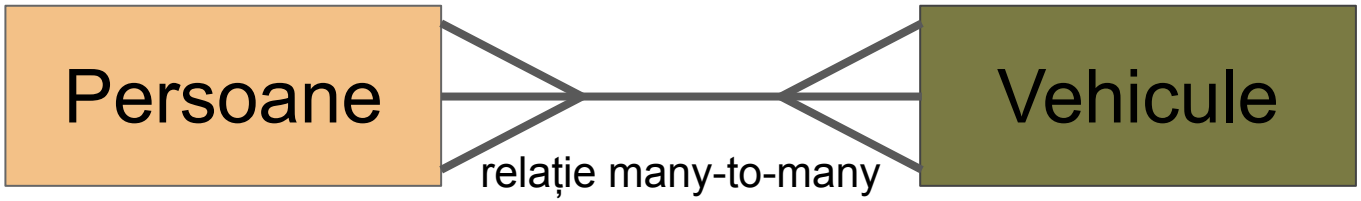
Detalii:

https://www.researchgate.net/publication/233755674_A_Survey_on_Reactive_Programming

O situație reală

Lulu conduce bicicletă și mașină, Judith conduce trotinetă și vapor, Aladin conduce trotinetă, role și bicicletă, Heidi conduce avion, role ai bicicletă





Cine conduce bicicleta?

Programare imperativă procedurală (javascript)

```
const conducatori = [
        { nume: "Lulu", conduce: ["bicicleta", "masina"] },
        { nume: "Judith", conduce: ["trotineta", "vapor"] },
        { nume: "Alladin", conduce: ["trotineta", "role", "bicicleta"] },
        { nume: "Heidi", conduce: ["avion", "role", "bicicleta"] }];
```

structura de date

```
var bicicleta = cineconduce("bicicleta");

print(bicicleta);

function cineconduce(vehicul) {
        let rezultate = [];
                for (i=0; i < conducatori.length; i++) {
                        for (j=0; j < conducatori[i].conduce.length; j++) {
                                if (conducatori[i].conduce[j] == vehicul) {
                                        rezultate.push(conducatori[i].nume);
                                }
                        }
                }
                return rezultate;
        }
}
```

https://onlinegdb.com/rJuzF3aLw

cod executabil format dintr-o funcție și un apel al acesteia din afara ei. Anumite limbaje de programare tratează orice parte a programului principal care nu se găsește într-o funcție ca făcând parte din funcția “main”

obs: **print** nu este o instrucțiune standard javascript, javascript nu are instrucțiuni de output, acestea depind de implementare, în acest caz Rhino

vehicule_procedural.js

Programare imperativă procedurală (c)

```c
#include <stdio.h>
#include <string.h>
```

Încărcare biblioteci de sistem

```c
#define MAX_CONDUCATORI 4
#define MAX_VEHICULE 3
```

Dimensionarea matricilor

```c
typedef struct {
   const char* nume;
   const char* conduce[MAX_VEHICULE];
   int nr_vehicule;
} Conducator;

Conducator conducatori[MAX_CONDUCATORI] = {
   {"Lulu", {"bicicleta", "masina"}, 2},
   {"Judith", {"trotineta", "vapor"}, 2},
   {"Alladin", {"trotineta", "role", "bicicleta"}, 3},
   {"Heidi", {"avion", "role", "bicicleta"}, 3}
};
```

Structura de date: definiția componenței acesteia și adăugarea datelor

```c
void cineconduce(const char* vehicul) {
   for (int i = 0; i < MAX_CONDUCATORI; i++) {
       for (int j = 0; j < conducatori[i].nr_vehicule; j++) {
           if (strcmp(conducatori[i].conduce[j], vehicul) == 0) {
               printf("%s\n", conducatori[i].nume);
           }
       }
   }
}
```

Funcția care caută informațiile necesare

```c
int main() {
   printf("Cine conduce bicicleta: \n");
   cineconduce("bicicleta");
   return 0;
}
```

Funcția principală a programului

https://onlinegdb.com/eZPrG1x0n

Programare imperativă obiectuală

Programarea orientată obiect încearcă să modeleze programul ca pe o serie de entități (obiecte) care interacționează între ele prin mesaje. Conceptual, paradigma este foarte ofertanta pentru ca face programul sa semene cu un ecosistem natural, ecosistem pe care creierul uman e obișnuit să îl înțeleagă.

Un obiect se obține în timpul rulării programului pe baza clasei corespunzătoare (ceea ce se programează sunt clasele, nu obiectele)

Obiectul este o abstracțiune informatică, posibilă datorită modului de organizare a limbajului și a compilatorului, care încapsulează comportamentul și starea.

- **Starea** este specificată prin **atribute** (culoare, greutate etc.)
- **Comportamentul** este specificat prin **metode** (calculeaza_media, schimba_culoarea etc)

Obiectele comunică între ele prin **mesaje**

Construcția unei aplicații într-un mediu orientat obiect începe înainte de scrierea codului, cu o etapă serioasă de proiectare, etapă în care se stabilesc obiectele, atributele și metodele lor.

Într-o fermă orientată obiect există un fermier orientat obiect și o vacă orientată obiect. Fermierul mulge vaca. Laptele este:

?
- obiectul lapte
- mesaj între fermier și vacă
  - ?
  - atașăm metoda “mulge” la obiectul fermier
  - atașăm metoda “da_lapte” la obiectul vacă

Programare imperativă obiectuală - exemplu

```
int main()
{
    Vehicul bicicleta{ "Bicicleta" };
    Vehicul trotineta{ "Trotineta" };
    Vehicul masina{ "Masina" };
    Vehicul vapor{ "Vapor" };
    Vehicul role{ "Role" };
    Vehicul avion{ "Avion" };
    Conducator lulu{ "Lulu" };
    Conducator judith{ "Judith" };
    Conducator aladin{ "Alladin" };
    Conducator heidi{ "Heidi" };
    lulu.addVehicul(bicicleta);
    lulu.addVehicul(masina);
    judith.addVehicul(trotineta);
    judith.addVehicul(vapor);
    aladin.addVehicul(trotineta);
    aladin.addVehicul(role);
    aladin.addVehicul(bicicleta);
    heidi.addVehicul(avion);
    heidi.addVehicul(role);
    heidi.addVehicul(bicicleta);

    cout << bicicleta << '\n';
    return 0;
}
```

Exemplu din pagină este incomplet, vedeți codul complet urmând linkul de mai jos sau în git.

Instanțierea obiectelor de tip Vehicul

Instanțierea obiectelor de tip Conducător

Adăugarea vehiculelor la Conducători folosind metoda addVehicul.

cout imprima bicicleta la consolă

Putem “imprima” un obiect?
In principiu nu, dar se poate “supraîncărca” operatorul <<

vehicule_obiectual.cpp

https://onlinegdb.com/HyvTXorwP

Programare declarativă cu DSL - Expresii regulate

inventat de https://en.wikipedia.org/wiki/Stephen_Cole_Kleene
prima implementare: https://en.wikipedia.org/wiki/Ken_Thompson

orice literă, majusculă sau nu, sau cifră, sau simbolurile ., _, %, -

o dată sau de mai multe ori

@

orice literă, majusculă sau nu, sau cifră, sau simbolurile ., _

o dată sau de mai multe ori

\.

orice literă, majusculă sau nu

de 2 până la 6 ori

[a-zA-Z0-9._%-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,6}

tot ce se află la stânga simbolului @

tot ce se află la dreapta simbolului @

Cineva a scris un email de pe adresa cracanel@memyself.com către adresele: pompiliu.chininescu@doctorul.ro, cufurescu_88@toilette.com.ro în care menționa dorința de a vinde un produs îndoielnic. Consider că acest abuz trebuie pedepsit cu tărie conform politicii de toleranță ZERO față de comunicări nesolicitate. Pentru aceasta, am trimis acest email si pe adresele: 1234567890@numarulcelmaimare.xxx, gentle-Man@iron-self.tv și Gladiather@caramela.tv. Dacă considerați necesar, vă rog sa trimiteți și către venerus@gineco.museum și carcalete@insecticid.co.jp și dacă e absolut necesar si către seful-cel-mai-mare@supercompanie.com.

Testare expresii regulate online:

https://regexr.com/

https://www.regextester.com/

ex_regexp_emails.php

https://onlinegdb.com/Hy36yAdDD

Programare declarativă cu DSL - Expresii regulate

Lulu conduce bicicleta si masina; Judith conduce trotineta si vapor; Aladin conduce trotineta role si bicicleta; Heidi conduce avion role si bicicleta.

([A-Z]\w+)

Parantezele rotunde marchează un grup de captură

Fragmentul va putea fi folosit ulterior

Unul sau mai multe caractere de tip cuvânt

cuvântul conduce

cuvântul bicicletă

; sau .

```
([A-Z]\w+)\sconduce[^;]*?bicicleta.*?(;|\.)
```

Orice grup de litere care începe cu o majusculă

un spațiu

orice caracter care nu este ; de oricâte ori (*) până la prima apariție a ceea ce urmează (?)

orice grup de litere (.) de oricâte ori (*) până la prima apariție a ceea ce urmează (?)

Pentru rezolvarea acestei probleme, este necesară utilizarea unei expresii regulate de înlocuire, nu doar una de identificare.

De aceea folosim replace si indicam $1 ca fiind primul grup de captură.

```
var mytext = 'Lulu conduce bicicleta si masina; Judith conduce trotineta si vapor; Aladin conduce trotineta role si bicicleta; Heidi conduce avion role si bicicleta.';

while ($text =~ /([A-Z]\w+)\sconduce[^;]*?bicicleta.*?(;|\.)/g) {
   print $1."\n";
}
```

vehicule_regexp.pl

https://onlinegdb.com/rJyud1sOv

Programare declarativă cu DSL - SQL

modelul relational: https://en.wikipedia.org/wiki/Edgar_F._Codd
implementare: https://en.wikipedia.org/wiki/Donald_D._Chamberlin si https://en.wikipedia.org/wiki/Raymond_F._Boyce

SQL - Structured Query Language este un limbaj utilizat pentru extragerea datelor stocate in bazele de date relaționale. Acestea au o structură aparte - datele sunt stocate în tabele care sunt conectate între ele prin relații precis definite.

persoane

ID	NUME
1	Lulu
2	Judith
3	Aladin
4	Heidi

persoane-vehicule

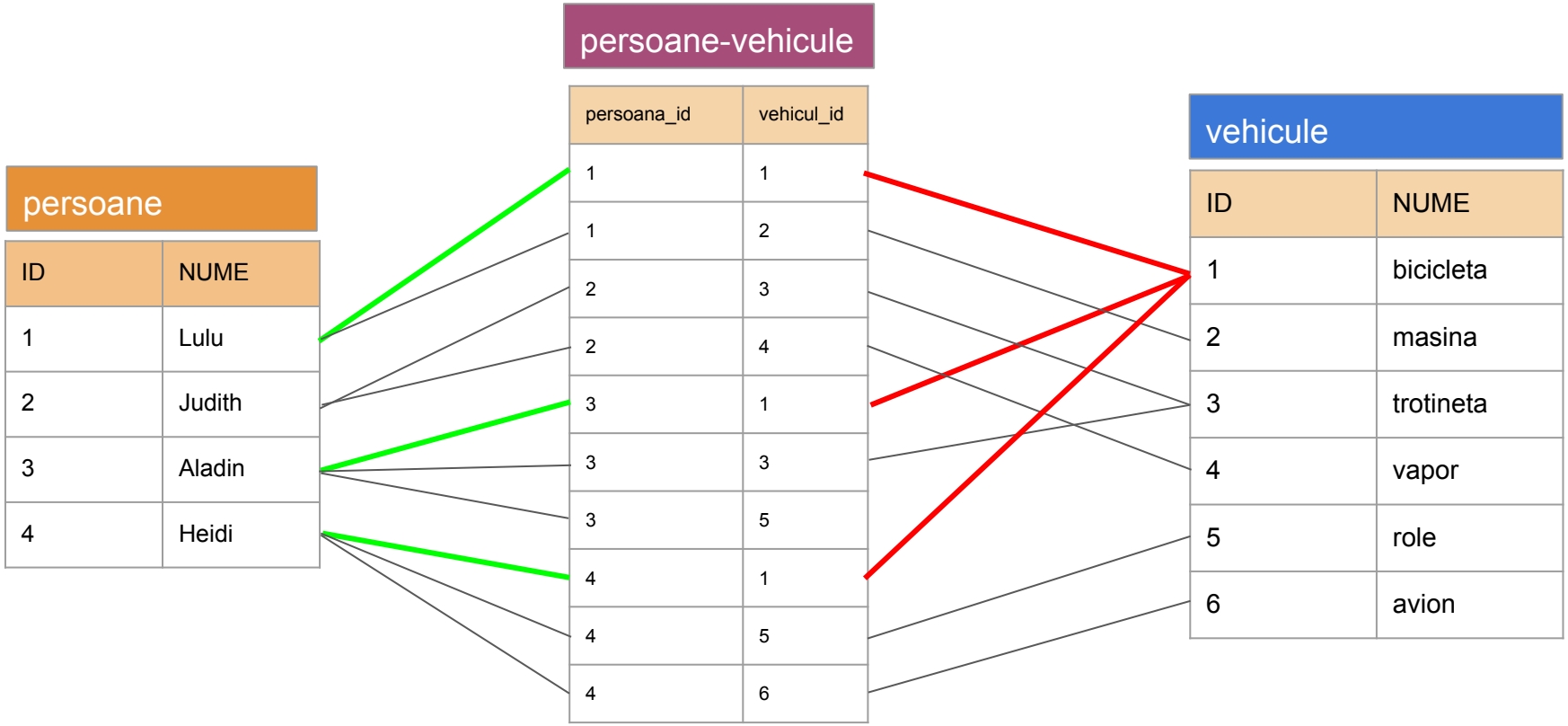
persoana	vehicul
1	1
1	2
2	3
2	4
3	1
3	3
3	5
4	1
4	5
4	6

vehicule

ID	NUME
1	bicicleta
2	masina
3	trotineta
4	vapor
5	role
6	avion

Lulu
Aladin
Heidi

conducatori.sql + conducatori_create_database.sql



SELECT persoane.nume from persoane
JOIN persoane_vehicule on persoane_vehicule.persoana_id=persoane.id
JOIN vehicule on vehicule.id=persoane_vehicule.vehicul_id
WHERE vehicule.nume='bicicleta'

Programare declarativa cu DSL - SQL

```
SELECT persoane.nume from persoane
JOIN persoane_vehicule on persoane_vehicule.persoana_id=persoane.id
JOIN vehicule on vehicule.id=persoane_vehicule.vehicul_id
WHERE vehicule.nume='bicicleta'
```

Echivalentul procedural

```
for (i = 0; i < persoane.length; i++) {
        print persoane[i].id, persoane[i].nume, persoane[i.prenume]
        for (j=0; j < orase.length; j++) {
                if (orase[j].id == persoane[i].oras) {
                        print orase[j].nume
                }
        }
}
```

SQL este cel mai utilizat limbaj pentru extragerea datelor structurate în modelul relational. Aproape orice aplicație cu care interacționați stochează datele într-o baza de date din care le extrage la solicitarea dvs.

Programare declarativă cu DSL - HTML + CSS

```
<!DOCTYPE html>
  <html lang="ro-RO">
    <head>
      <meta charset="UTF-8" />
      <link rel="stylesheet" type="text/css" href="//www.utm.ro/wp-content/cache/wpfc-minified/10lvt9sj/1fmj.css"
media="all"/>
      <title>Universitatea Titu Maiorescu | Grad de încredere ridicat</title>
      <meta name="description" content="Facultăți: Psihologie, Drept, Medicină, Medicină Dentară, Farmacie, Științe Sociale,
Politice și Umaniste, Ştiinţe Economice şi Informatică." />
      <meta name="robots" content="index, follow" />
    </head>
    <body data-rsssl=1 class="home page-template-default page page-id-9 fusion-image-hovers fusion-pagination-sizing
wpb-js-composer js-comp-ver-6.4.1 vc_responsive">
      <a name="top" style="height:0; position:absolute; top:0;" id="top-anchor"></a>
      <div id="body-wrap">
        <div id="wrap">
          <header>
            <div id="top-nav" class="dark-div nav-style-2">
              <nav class="navbar navbar-inverse main-color-1-bg" role="navigation">
                <ul class="nav navbar-nav hidden-xs">
                 <li id="nav-menu-item-wpml-ls-5-ro" class="main-menu-item menu-item-depth-0 menu-item-language
menu-item-language-current menu-item wpml-ls-slot-5 wpml-ls-item wpml-ls-item-ro wpml-ls-current-language
wpml-ls-menu-item wpml-ls-first-item menu-item-type-wpml_ls_menu_item menu-item-object-wpml_ls_menu_item"><a
title="Română" href="https://www.utm.ro" class="menu-link  main-menu-link"><img onload="Wpfcll.r(this,true);"
src="https://www.utm.ro/wp-content/plugins/wp-fastest-cache-premium/pro/images/blank.gif" class="wpml-ls-flag"
data-wpfc-original-src="https://www.utm.ro/wp-content/plugins/sitepress-multilingual-cms/res/flags/ro.png" alt="blank"><span
class="wpml-ls-native" lang="ro">Română</span> </a></li>
                 <li id="nav-menu-item-wpml-ls-5-en" class="main-menu-item menu-item-depth-0 menu-item-language
menu-item wpml-ls-slot-5 wpml-ls-item wpml-ls-item-en wpml-ls-menu-item wpml-ls-first-item menu-item-type-wpml_ls_menu_item
menu-item-object-wpml_ls_menu_item"><a title="English" href="https://www.utm.ro/en" class="menu-link
main-menu-link"><img onload="Wpfcll.r(this,true);"
src="https://www.utm.ro/wp-content/plugins/wp-fastest-cache-premium/pro/images/blank.gif" class="wpml-ls-flag"
data-wpfc-original-src="https://www.utm.ro/wp-content/plugins/sitepress-multilingual-cms/res/flags/en.png" alt="blank"><span
class="wpml-ls-native" lang="en">English</span> </a></li>
                 </ul>
```

Header - nu este direct vizibil, conține instrucțiuni pentru motoarele de căutare, fișiere auxiliare, metadate.

DIV - Crează un container, ca un fel de partiție care este afișată conform specificațiilor aflate in proprietățile acestuia

UL - Crează o listă de elemente

HTML + CSS sunt declarative dar nu sunt chiar programare. DSL-ul nu este complet. Problema inversării listelor nu se poate rezolva în HTML + CSS acum. De aceea este nevoie încă de Javascript, pentru a suplimenta partea de programare.

Programare declarativă - Azure ARM si Terraform

arm

```
[
{
 "name": "ipAddressName",
 "type": "Microsoft.Network/publicIpAddresses",
 "apiVersion": "2020-08-01",
 "location": "westeurope",
 "properties": {
   "publicIpAllocationMethod": "Static"
  }
},
{
 "type": "Microsoft.Network/virtualNetworks",
 "name": "MainVnet",
 "apiVersion": "2024-01-01",
 "location": "westeurope",
 "properties": {
   "addressSpace": {
     "addressPrefixes": "10.0.0.0/16"
   },
   "subnets": [
    {
     "name": "defaultSubnet",
     "properties": {
      "addressPrefix": "10.0.0.0/24"
     }
    }
   ]
  }
},
]
```

terraform

```
resource "azurerm_virtual_network" "example" {
 name              = local.virtual_network_name
 address_space     = var.address_prefixes
 location          = azurerm_resource_group.example.location
 resource_group_name  = azurerm_resource_group.example.name
}

resource "azurerm_subnet" "example" {
 name                   = "internal"
 resource_group_name    = azurerm_resource_group.example.name
 virtual_network_name   = azurerm_virtual_network.example.name
 address_prefixes       = ["10.0.0.0/24"]
}

resource "azurerm_public_ip" "example" {
 name                  = local.public_ip_address_name
 location              =
azurerm_resource_group.example.location
 allocation_method     = "Dynamic"
 resource_group_name   = azurerm_resource_group.example.name
 sku = "Basic"
}
```

Programare funcțională

calcul lambda: https://en.wikipedia.org/wiki/Alonzo_Church
lisp: https://en.wikipedia.org/wiki/John_McCarthy_(computer_scientist)

Programarea funcțională este o paradigmă în care un program este compus prin **apelarea și compunerea funcțiilor**.

Din punct de vedere al codului, funcțiile sunt tot subrutine care au o serie de proprietăți suplimentare, care provin din faptul că sunt considerate "**componente de primă clasă** (first class citizens)" - elemente care suporta absolut toate operațiile disponibile. Astfel, față de o subrutină obișnuită, o funcție poate:

- să fie atribuită unui nume (ca o variabilă)
- să fie returnată dintr-o altă funcție
- să fie argument al unei funcții
- să poată fi verificată din punct de vedere al egalității

În limbajele de programare imperative, tipurile de date standard (int, etc.) sunt aproape întotdeauna componente de primă clasă.

În majoritatea limbajelor vechi (inclusiv C) tablourile NU sunt componente de primă clasă (de ex. în C nu pot fi trimise către o funcție sau returnate, aceste operații se realizează cu ajutorul pointerilor).

Programare funcțională pură - toate funcțiile NU au efecte secundare. Aceasta înseamnă că o funcție NU trebuie să modifice nimic în afara ei, nici o variabilă globală, sau trimisă prin referință. Astfel, starea generală a programului NU trebuie să se modifice în nici un fel în urma apelării unei funcții.

Programare funcțională (javascript)

```
const conducatori = [
        { nume: "Lulu", conduce: ["bicicleta", "masina"] },
        { nume: "Judith", conduce: ["trotineta", "vapor"] },
        { nume: "Alladin", conduce: ["trotineta", "role", "bicicleta"] },
        { nume: "Heidi", conduce: ["avion", "role", "bicicleta"] }];
```

Programare imperativă

```
let rezultate = [];
const cineconduce = "bicicleta";
for (i=0; i < conducatori.length; i++) {
 for (j=0; j < conducatori[i].conduce.length; j++) {
  if (conducatori[i].conduce[j] == cineconduce) {
   rezultate.push(conducatori[i].nume);
  }
 }
}
console.log(rezultate);
```

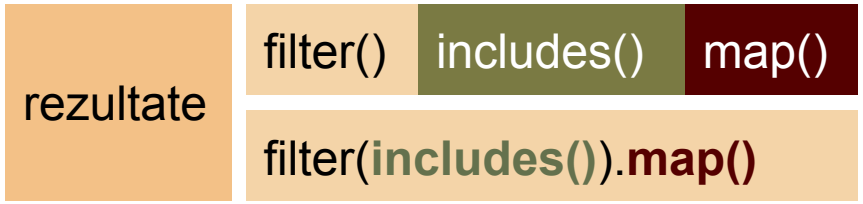
https://onlinegdb.com/YhLLkfrCu

Programare funcțională

```
const cineconduce = "bicicleta";

const rezultate = conducatori.filter(conducator =>
conducator.conduce.includes(cineconduce)).map(c
onducator => conducator.nume);

console.log(rezultate);
```

https://onlinegdb.com/pWPizhXQ3z

Limbaje de programare funcțională “pure” sunt: Haskel, Erlang



Javascript are multe elemente de programare funcțională. Dar, ca în orice domeniu, și în programare există obsesii majore pentru puritatea anumitor concepte. Există colective în care a numi Javascript “un limbaj funcțional” poate duce la emoții intense.

Programarea logică

Un program este constituit dintr-o lista de axiome care se împart în fapte și clauze

Fiecare axioma este reprezentată printr-o clauză HORN

rezultat :- condiție1 și condiție2 și condiție3….

condiție1 si condiție2 și condiție3 implică rezultat

rezultatul se numește SCOP. El este adevărat dacă toate condițiile de după semnul :- (numite subscopuri) sunt adevărate.

Un program logic pornește de la o bază de cunoștințe care reprezintă o serie de propoziții adevărate exprimate sub forma clauzelor HORN.

Dacă baza de cunoștințe este completă și coerentă, programul poate să răspundă la întrebări derivate pe baza inferenței logice.

Programul în sine reprezintă un scop a cărui valoare de adevăr trebuie determinată.

Programarea logică - Exemplu

Knowledge base

https://onlinegdb.com/JoMkzaVjc

Clauza compusă

```
conduce(lulu, [bicicleta,masina]).
conduce(judith, [trotineta,vapor]).
conduce(aladin,[trotineta,role,bicicleta]).
conduce(heidi,[avion,role,bicicleta]).

cineconduce(Pers,Objct) :- conduce(Pers, Obiecte), member(Objct, Obiecte).
```

CLAUZĂ COMPUSĂ	cineconduce(Pers,Objct) :- conduce(Pers, Obiecte), member(Objct, Obiecte).
SCOP	cineconduce(Pers,Objct)
COND 1	conduce(Pers, Obiecte)
COND 2	member(Objct, Obiecte)

Persoana Pers conduce Objct dacă:

conduce Obiecte (căutare în liste)

din Obiecte face parte Objct

Rularea programului

?- cineconduce(X,bicicleta).
X = lulu ;
X = aladin ;
X = heidi.

Întrebarea cineconduce(X,bicicleta) va parcurge toată baza de informații si va verifica veridicitatea clauzei compuse “cineconduce”. Va răspunde doar în cazul în care clauza compusă este adevărată.

Utilizari:

Orarul unei facultăți. , Planificarea călătoriilor, rutare. , Partajarea resurselor comune, Analiza limbajelor standardizate, Sisteme decizionale, Business intelligence

Datalog - subset al PROLOGULUI care este utilizat pentru baze de date (Datomic este un serviciu de baze de date distribuite care folosește datalog)

Imperativ vs. Declarativ

Programarea imperativă

- Descrie o secvență de evenimente care se succed în timp.
- Solicită prezența unui executor (compilator, mașină virtuală)
- Secvența poate rula sincron sau asincron
- Cerințe reduse din partea mediului de execuție

Programarea declarativă

- Descrie starea unui sistem, astfel încât să proceseze datele de intrare după nevoie
- Solicită prezența unui configurator (control plane, procesor, motor grafic)
- Sistemul reacționează la prezența datelor de intrare aplicând rețeta configurată.
- Cerințe semnificative din partea mediului de execuție

X - as Code

- **Infrastructure as Code**
  - **Descrie**: mașini virtuale, rețele, stocare, etc.
  - **Executor**: Azure, AWS, Google Cloud
  - **Limbaje**: ARM, Terraform, Pulumni (SDK pentru Python, Java, etc.)
- **Platform as Code**
  - **Descrie**: aplicații, baze de date, servicii
  - **Executor**: Kubernetes, Docker swarm, Cloud Foundry
  - **Limbaje**: YAML, JSON, HCL, Pulumni (SDK pentru Python, Java, etc.)
- **Configuration as Code**
  - **Descrie**: pachete software, servicii, setări ale aplicațiilor
  - **Executor**: Managere de configurație (Ansible, Puppet, Chef)
  - **Limbaje**: YAML, Puppet DSL, Chef DSL, etc.
- **Network as Code**
  - **Descrie**: configurare a dispozitivelor de rețea, reguli, firewall
  - **Executor**: Dispozitive configurabile de rețea
  - **Limbaje**: YAML, JSON, Cisco DSL, Junos, etc.
- **Diagrams as Code**
  - **Descrie**: Diagrame, scheme logice, diagrame de secvență
  - **Executor**: Diverse motoare grafice - PlantUML, Graphviz, draw.io
  - **Limbaje**: XML, Plant UML DSL, Structurizr DSL, etc.
- **Documents as Code**
  - **Descrie**: Documente (Titluri, subtitluri, referințe)
  - **Executor**: Procesoare specifice, pdflatex, Asciidoctor, Pandoc
  - **Limbaje**: Markdown, Latex, AsciiDoc

Programarea multiparadigmă (duh…)

Dezvoltarea instrumentelor de programare (nu doar compilatoare ci și întregul ansamblu de biblioteci pe care ajunge să le folosească un programator în dezvoltarea unei aplicații complexe) determină **tranziția dinspre imperativ spre declarativ.**

Multe sisteme pre-programate conțin componente flexibile, cu multe funcționalități, care pot fi apelate în programele noastre. Apelarea acestora este de cele mai multe ori **declarativă**.

Orice aplicație ceva mai complicată **va avea zone declarative semnificative** - fie încărcarea componentelor externe, fie interacțiunea cu sisteme de stocare (ex: SQL).

De asemenea, majoritatea aplicațiilor vor avea **zone funcționale**, fie reacționând la fluxul de date în stilul SAX, fie construind zone asincrone, sisteme înlănțuite de promisiuni, etc.

Orice demers serios de programare este multiparadigmă. Ceea ce este important este să detectați corect paradigma în care vă aflați în momentul respectiv. Dacă scrieți un sistem de configurare a unui component extern, nu căutați locul în care puteți sa scrieți instrucțiuni de ciclare, etc.

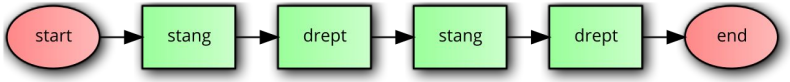
Algoritmi

Structuri algoritmice

https://code2flow.com/

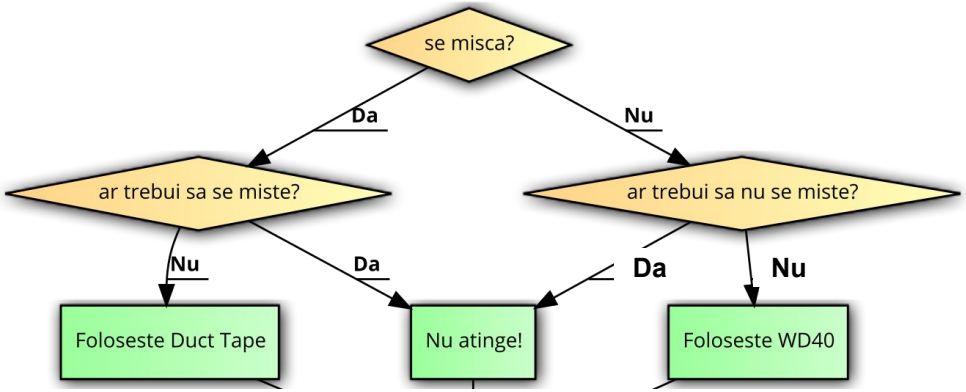
secvență

Secvența reprezintă o succesiune de pași care se execută în ordinea în care au fost scriși în cod.



selecție

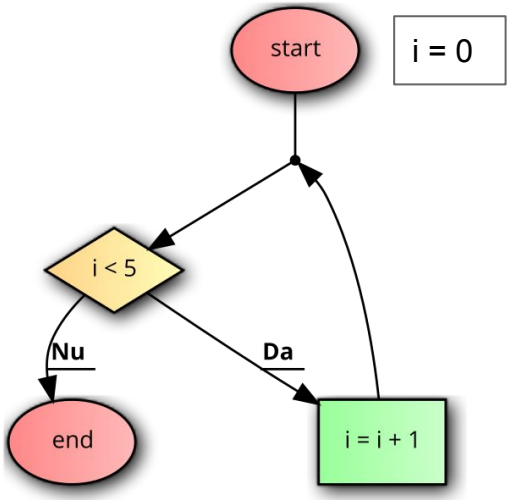
Selecția reprezintă o decizie care duce la execuția unor secvențe diferite în funcție de veridicitatea unei condiții predefinite.



If - then - elsif - else
switch - case

repetiție

Repetiția repetă o secvență o dată sau de mai multe ori până la îndeplinirea unei condiții predefinite.



for - end
do - while
while - end

SECVENȚE

Secvența reprezintă o succesiune de pași care se execută în ordinea în care au fost scriși în cod.

```
a = 12
b = x + y / z
write "ceva"
read N
```

```
write "principal_amt, rate and time"
read principal_amt, rate, time
interest = (principal_amt * rate * time) / 100.0
write "Amount", principal_amt
write "Rate", rate
write "Time", time
write "Interest, interest
```

SELECȚIE: if - then - else if - else

pseudocod

```
write "introduceti N"
read N
if N > 0 then
    write "Numarul e mai mare decat zero"
else if N == then 0
    write "Numarul e zero"
else
    write "Numarul e mai mic decat zero"
end if
```

Selecția reprezintă o decizie care duce la execuția unor secvențe diferite în funcție de veridicitatea unei condiții predefinite.

c

if_else_zero.c

```c
#include <stdio.h>
int main()
{
    int N;
    printf("give me a Number\n");
    scanf("%i",&N);
    if (N > 0) {
        printf("Numarul e mai mare decat zero\n");
    } else if (N == 0) {
        printf("Numarul e zero\n");
    } else {
        printf("Numarul e mai mic decat zero\n");
    }
    return 0;
}
```

https://onlinegdb.com/BJSuqIN6m

SELECȚIE: switch - case

Selecția reprezintă o decizie care duce la execuția unor secvențe diferite în funcție de veridicitatea unei condiții predefinite.

pseudocod

```
char gr = 'B';
case on grade
  case 'A' :
    write "Excelent!", gr
    break;
  case 'B' :
    write "Close, but no cigar", gr
    break;
  case 'C' :
    write "On the top of gauss hat", gr
    break;
  case 'D' :
    write "Still in school, not for long", gr
    break;
  case 'F' :
    write "you need a job\n", gr
    break;
  default :
    write "Invalid grade", gr
  end case
```

c

switch.c

```c
#include <stdio.h>

int main () {
  char gr = 'B';
  switch(gr) {
     case 'A' :
        printf("%c: Excelent!\n", gr );
        break;
     case 'B' :
        printf("%c: Close, but no cigar\n", gr );
        break;
     case 'C' :
        printf("%c: On the top of gauss hat\n", gr);
        break;
     case 'D' :
        printf("%c: Still in school, not for long\n",
gr);
        break;
     case 'F' :
        printf("%c: you need a job\n", gr);
        break;
     default :
        printf("%c: Invalid grade\n", gr);
  }
  return 0;
}
```

https://onlinegdb.com/r1XEOINa7

```
B: Close, but no cigar
```

REPETIȚIE: calculul mediei

bucla while

pseudocod

```
suma = 0
cnt = 1
avg = 0
write "give me N"
read N
while cnt <= N
    write "Numarul nr.", cnt
    read tmp
    sum = sum + tmp
    avg = sum / cnt
    cnt = cnt + 1
end while
average = sum / i
write "media este: ", average
```

```
while (cnt <= N) {
    ….
}
```

c

average-while.c

```c
#include <stdio.h>

int main()
{
    int sum=0, count=1, N=0, tmp=0;
    float avg;
    printf("give me N\n");
    scanf("%i",&N);
    while (count < N+1) {
        printf("Numarul nr. %i\n", count);
        scanf ("%i",&tmp);
        sum = sum + tmp;
        avg = (float)sum / count;
        count = count+1;
    }

    printf("media este: %f\n",avg);
    return 0;
}
```

https://onlinegdb.com/BkfimULPP

```
give me N
3
Numarul nr. 1
12
Numarul nr. 2
17
Numarul nr. 3
2
media este: 10.333333
```

Buclele de tip while sunt bucle care se executa atâta timp cât condiția din paranteză este adevărată. Verificarea se face înainte de execuția blocului de cod din bucla.

REPETIȚIE: Calculul mediei
bucla for

pseudocod

```
suma = 0
cnt = 1
avg = 0
write "give me N"
read N
for cnt = 1, N, 1
    write "Numarul nr.", cnt
    read tmp
    sum = sum + tmp
    avg = sum / cnt
end for

write "media este: ", average
```

c

average-for.c

```c
#include <stdio.h>

int main()
{
    int sum, count, N, tmp;
    float avg;
    printf("give me N\n");
    scanf("%i",&N);
    for (count = 1; count <= N; count++) {
        printf("Numarul nr. %i\n", count);
        scanf ("%i",&tmp);
        sum = sum + tmp;
        avg = (float)sum / count;
    }

    printf("media este: %f\n",avg);
    return 0;
}
```

```
for (cnt = 1; cnt <= N; cnt++) {
    ….
}
```

https://onlinegdb.com/rk6qFI4pX

Buclele de tip for sunt bucle la care numărul de iterații este declarat în momentul execuției buclei.

În pseudocod acestea se pot scrie și sub forma repeat N

Funcții, proceduri, subrutine

Procedurile reprezintă **mici subprograme** care pot fi separate de corpul programului principal. Ce anume intră într-o procedură reprezintă o decizie de proiectare a programului. În anumite limbaje de programare acestea sunt numite proceduri, în altele funcții, în altele subrutine.

C nu are un cuvânt special pentru definirea procedurilor, acestea sunt specificate prin nume, argumente precum și prin tipul datei pe care îl returnează. Mai multe limbaje de programare urmează același model (Java, C#, Python, Ruby, Scala). Alte limbaje folosesc cuvinte rezervate (instrucțiuni) pentru declararea unei proceduri.

Javascript	function	function solveWorldHunger(a, b, c)
Pascal	procedure	procedure solveWorldHunger(a, b, c: integer;)
Basic	sub	sub solveWorldHunger(a,b,c)
Php	function	function solveWorldHunger(a, b, c)
Fortran	subroutine	subroutine solveWorldHunger(a,b,c)
Perl	sub	sub solveWorldHunger
Lisp	defun	defun solveWorldHunger(a,b,c)

Funcții, proceduri, subrutine

Orice limbaj de programare de nivel înalt are o serie de funcții predefinite care pot fi utilizate direct în cod.

Orice limbaj de programare permite definirea propriilor funcții.

Împărțirea unui program în funcții ajută la:

1. Ușurința înțelegerii codului de către oameni
2. Rapiditatea scrierii codului în cazul operațiilor repetitive (ex: calculul radicalului unui număr, care poate apărea de foarte multe ori într-un program).
3. Depanarea mai ușoară - în cazul în care există o greșeală într-o funcție, ea trebuie corectată într-un singur loc.
4. Reutilizarea codului - funcțiile pot fi copiate dintr-un program în altul sau chiar grupate în biblioteci care pot fi refolosite.
5. Posibilitatea proiectării în etape a unui program complex.
6. Executia algoritmilor recursivi

Toate fragmentele de cod C afișate mai sus încep cu linia `#include <stdio.h>`. Această linie anunță compilatorul că avem intenția de a utiliza funcții definite în biblioteca **stdio**, bibliotecă ce însoțește compilatoarele C.

Proceduri, subrutine - suport hardware

Programarea procedurală implică utilizarea procedurilor pentru a **separa** fragmente de cod care pot fi izolate. Rezultă un cod modular care este **mai ușor de citit**, **mai ușor de depanat**, de multe ori **mai scurt** (sunt multe fragmente de cod care se execută de multe ori).

Uneori există colecții de proceduri care se utilizează des și care sunt caracterizate de intrări și ieșiri foarte clare. Acestea sunt de obicei grupate în **biblioteci (libraries)** care sunt utilizate în timpul programării.

Principala provocare pusă în fața unui programator nu este aproape niciodată sintaxa limbajului de programare pe care îl folosește ci **cunoașterea operațională a colecției de biblioteci** pe care trebuie să le utilizeze.

In cazul C, biblioteci comune sunt: stdio, string, time, math, limits, threads, stdlib, ctype

Programarea în limbaj de asamblare poate fi procedurală. Prin existența instrucțiunilor de tip JUMP, diferite fragmente de cod pot fi poziționate în memorie la adrese precise iar procesorul poate fi instruit să “sară” de la una la alta.

În plus, registrele procesorului pot fi utilizate pentru a reține adresa procedurii care a apelat procedura curentă, permițând reîntoarcerea.

Prin combinația utilizării registrelor, a instrucțiunilor de salt și a modului de acces al memoriei, compilatoarele pot crea un model special de execuție pentru programele procedurale.

Programarea procedurală are suport hardware!

Limbajul de programare C

Limbajul de programare C

- Un limbaj cu scop general
- Un limbaj care folosește în principal programarea imperativă, structurată (procedurală)
- Tipurile de variabile sunt declarate și verificate înainte de compilare.

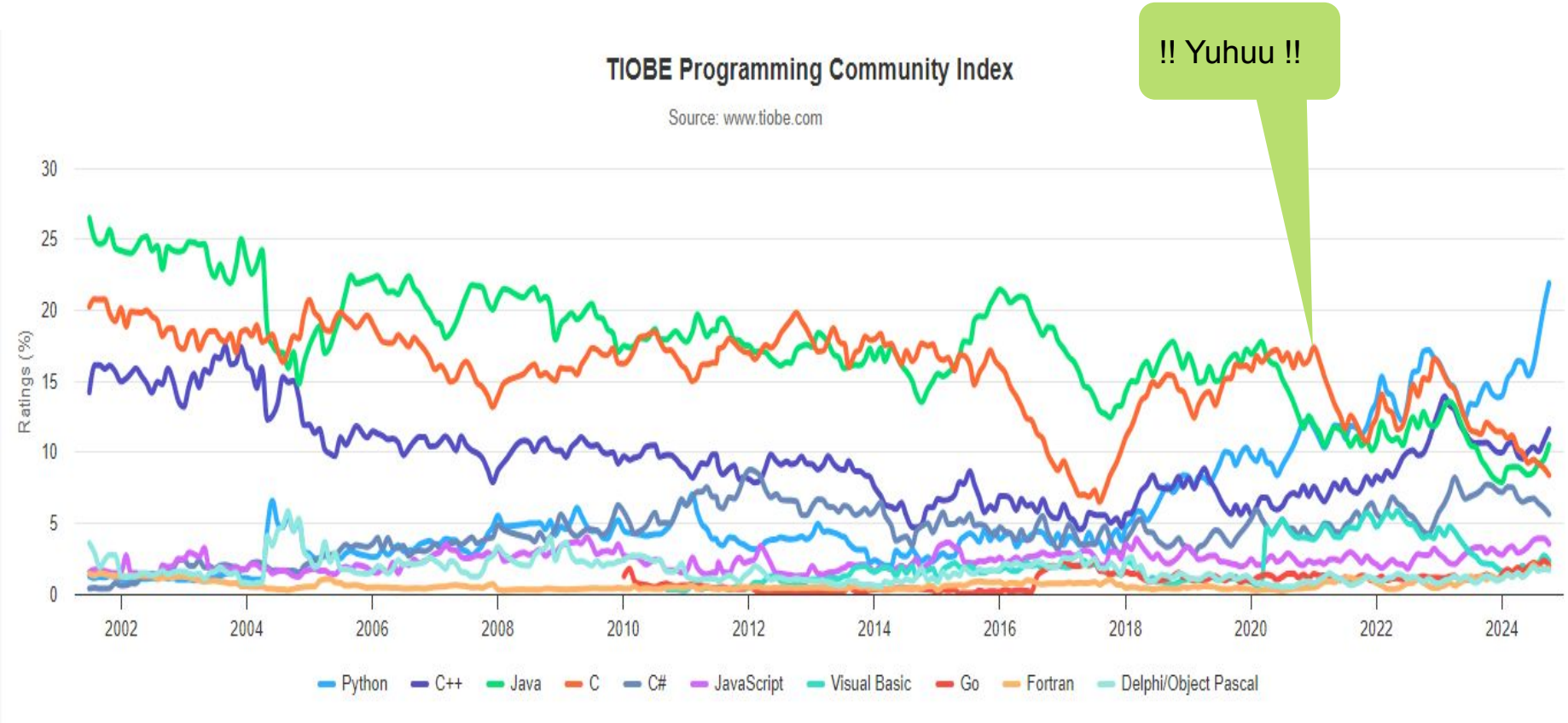
- Un limbaj relativ simplu, destul de apropiat de limbajul de asamblare
- Un limbaj robust care permite scrierea unui program oricât de complex.
- Programele scrise în C sunt de obicei rapide și eficiente (depinde de programator)
- Un limbaj extrem de portabil (codul poate fi ușor mutat de pe o arhitectură pe alta)
- Un limbaj care se poate extinde prin gruparea unor funcții în biblioteci.
- Un limbaj care permite accesul direct la memorie (**programmer driven memory management**).

În utilizarea curentă, programele scrise în C sunt cele care sunt apropiate de hardware: nuclee ale sistemelor de operare, driverele diverselor componente hardware, nucleele jocurilor care folosesc accelerații hardware, software care rulează pe sisteme automate… etc.

De ce să învățăm C?

C este unul dintre cele mai populare limbaje de programare

Tiobe măsoară popularitatea în motoarele de căutare, nu anunțurile de angajare!!



https://www.tiobe.com/tiobe-index/

De ce să cunoaștem C?

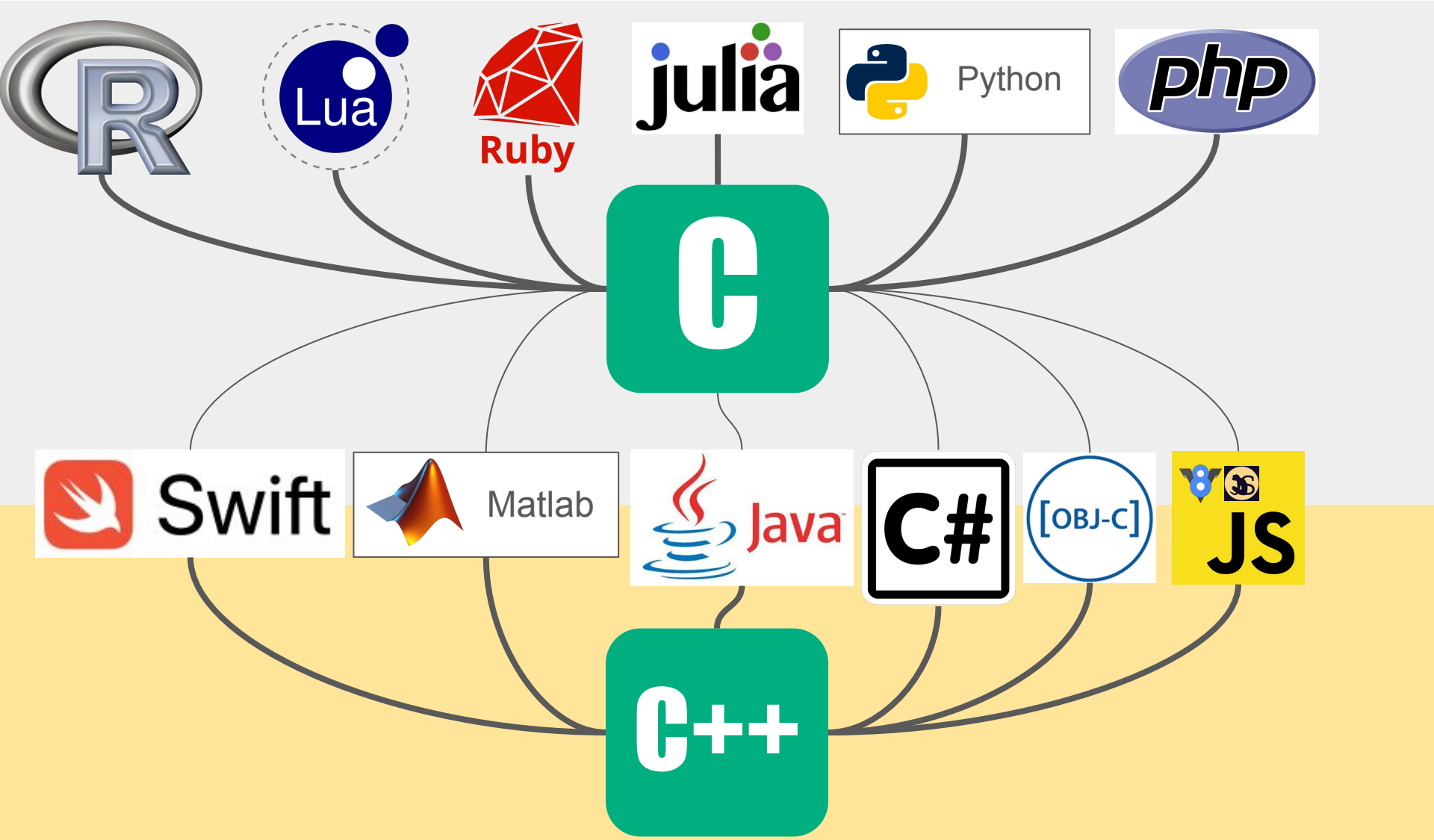
- Este rezonabil de popular, există în continuare cerere serioasă pentru programatori C.
- Programarea de sistem se face în C. Dacă există sisteme care permit programarea de sistem în alte limbaje, aproape de fiecare dată acestea se bazează pe un interpretor care traduce celelalte limbaje în C. Sau un compilator specializat care a fost scris în C.
- Programele scrise în C depășesc în performanțe programele scrise în multe alte limbaje.

De ce să începem cu C?

- Sintaxa C-ului a stat la baza multor altor limbaje de programare (Java, C++ C#, perl, php, javascript… etc.)
- La nivel introductiv, nu se pot studia caracteristicile avansate ale altor limbaje (C#, C++, Java).
- Fiind un limbaj de nivel mediu, este un instrument didactic bun pentru dezvoltarea unei gândirii corecte.

De ce să cunoaștem C?

C este limbajul în care este scris limbajul tău de programare (Alan Cox)?



C este peste tot

Trimitere mesaj whatsap de pe Android

Aplicația WhatsApp este în mare măsură Java dar folosește secvențe de cod C/C++ prin JNI pentru comunicare și altele.

```
fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
sa.sin_family = AF_INET; sa.sin_port =
htons(443); sa.sin_addr.s_addr =
inet_addr(server_ip);
connect(fd, (struct sockaddr*)&sa, sizeof sa);
send(fd, buf, len, 0);
close(fd);
```

Model AI

Orice model AI ajunge mai devreme sau mai târziu la inmulțiri de matrici, de obicei folosind bibliotecile CUDA.

```
for (t = 0; t < (K+T-1)/T; ++t) {
 As[ty][tx] = A[row*K + t*T + tx];
 Bs[ty][tx] = B[(t*T + ty)*N + col];
  __syncthreads();
 for (k = 0; k < T; ++k) acc = fmaf(As[ty][k],
Bs[k][tx], acc);
 __syncthreads();
}
C[row*N + col] = acc;
```

Vizualizarea unui video Youtube pe majoritatea browserelor

separarea fluxului audio de cel video, decodificare, trimiterea către GPU a cadrului video

```
while (av_read_frame(fmt, &pkt) >= 0) {
 if (pkt.stream_index == vsi) {                          //vsi = video stream index
   avcodec_send_packet(vdec, &pkt);                      //trimite pachetul catre decodorul
video
   while (avcodec_receive_frame(vdec, vf) == 0) {
     pts_sec = vf->best_effort_timestamp * av_q2d(vst->time_base);  //calculeaza poziia în timp
     sws_scale(sws, (const uint8_t * const)vf->data, vf->linesize, 0, vdec->height,
               rgb_planes, rgb_stride);                  // conversia cadrului YUV420 → RGB
     glBindTexture(GL_TEXTURE_2D, tex);              // specifică obiectul de tip textură curent
     glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, W, H, GL_RGBA, GL_UNSIGNED_BYTE, rgb_planes[0]);
     schedule_video_present(pts_sec);                  // vsync/clock sync
   }
 }
 av_packet_unref(&pkt);
}
```

C este peste tot

Programare în php

```php
<?php
$x = 5;
if ($x > 3) {
   $y = $x * 2;
}
?>
```

```
ZVAL_LONG(CV(0), 5); /* $x = 5; */

cond = (Z_TYPE_P(CV(0)) == IS_LONG && Z_LVAL_P(CV(0)) > 3); /* if ($x > 3) ...
*/
if (!cond) goto L_end;

ZVAL_LONG(&lit_2, 2); /* $y = $x * 2; */
fast_multiply_function(EX_VAR(0), CV(0), &lit_2);
ZVAL_COPY_VALUE(CV(1), EX_VAR(0));
L_end: /* end */
```

Afișarea unui site în Chrome (fragment din procesorul CSS flexbox, C++)

```
bool FlexLine::ResolveFlexibleLengths() {
 ...
 for (size_t i = 0; i < line_items.size(); ++i) {
   FlexItem& flex_item = line_items[i];
   if (flex_item.frozen) continue;
   LayoutUnit child_size = flex_item.flex_base_content_size;
   double extra_space = 0;

   if (remaining_free_space > 0 && total_flex_grow > 0 && flex_sign == kPositiveFlexibility) {
     extra_space = remaining_free_space *
                   flex_item.style.ResolvedFlexGrow(flex_box_style) / total_flex_grow;
   } else if (remaining_free_space < 0 && total_weighted_flex_shrink > 0 &&
              flex_sign == kNegativeFlexibility) {
     extra_space = remaining_free_space *
                   flex_item.style.ResolvedFlexShrink(flex_box_style) *
                   flex_item.flex_base_content_size / total_weighted_flex_shrink;
   }  ...   }  ...  }
```

Hello World

```c
#include <stdio.h>
int main()
{
  printf("Hello World");
  return 0;
}
```

În mod tradițional, primul exemplu într-un limbaj de programare este un program care tipărește în spațiul de ieșire expresia “Hello World”.

Tradiția a început în cartea **A Tutorial Introduction to the Language B** scrisă de Brian Kernighan, cel care urma să scrie și prima carte de programare în C, împreună cu unul dintre creatorii C, Dennis Ritchie

```c
#include <stdio.h>
```

Limbajul C nu conține instrucțiuni de intrare sau ieșire în nucleul său, operațiile respective sunt implementare în biblioteci dependente de sistem care însoțesc compilatoarele.

```c
int main()
{
  return 0;
}
```

Programele C sunt obligatoriu împărțite în funcții. Orice program trebuie să aibă cel puțin o funcție, obligatoriu numită **main**, care se execută prima automat. Cele două paranteze care însoțesc funcția main reprezintă un spațiu pentru argumente

```c
printf("Hello World");
```

printf este una dintre funcțiile definite în biblioteca stdio.h și este utilizată pentru imprimarea variabilelor la standard output.

```c
int main(int argc, char **argv)
{
   printf("Argc, %d\n", argc);
   printf("Argv, %c\n", argv[0]);
   printf("Argv, %c\n", argv[1]);
   return 0;
}
```

https://onlinegdb.com/B1r_XYEam

hello_args.c

```
> ./hello_args ala
Numar argumente, 2
Argumentul 0, ./hello_args
Argumentul 1, ala
```

Declarația completă a funcției main poate adăuga două argumente: numărul de argumente și un șir de argumente care au fost date programului la rulare.

Structura unui program C

```c
/**
* Program:
* Autor: sorin
* descriere: Program care calculeaza aria cercului
**/

#include <stdio.h>
#include <stdlib.h>

#define PI 3.141

int putere(int m, int n);

int main(int argc, char *argv[])
{
  float rad = 0;
  float area = 0;
  if( argc == 2 ) {
     rad = atof(argv[1]);
     area = PI * putere(rad, 2);
     printf("Aria cercului este %f\n", area);
  } else {
     // numarul de argumente de la linia de comanda e gresit
     printf("Numar gresit de argumente.\n");
  }
   return 0;
}

int putere(int m, int n)
{
  int i, p=1;
  for (i=1; i<=n; i++) {
      p = p * m;
  }
  return p;
}
```

Documentație

Include

Definiții

Prototipuri ale funcțiilor

Funcția principală

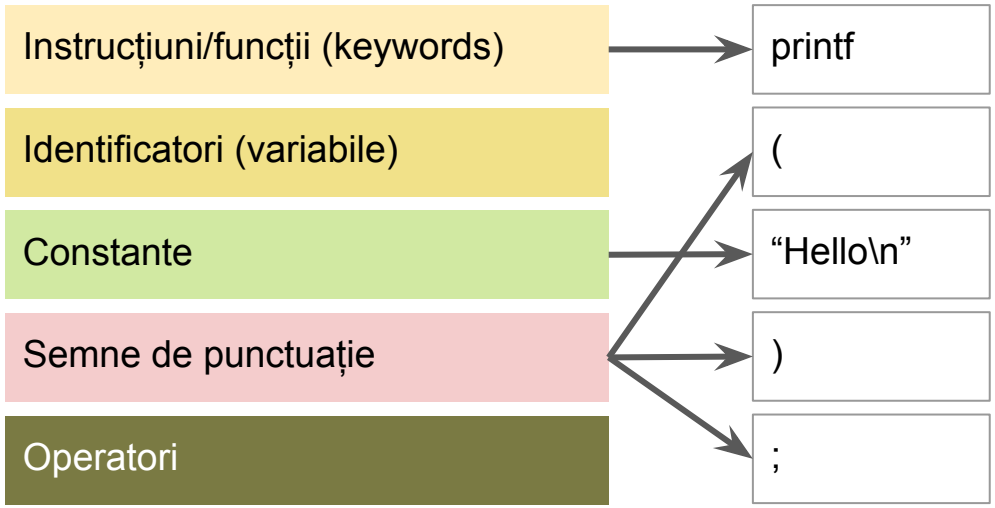
Altă funcție

https://embeddedartistry.com/blog/2019/04/08/a-general-overview-of-what-happens-before-main/

Elemente de sintaxă

Orice program C este văzut de compilator ca fiind format din fragmente (tokens) care pot fi de 5 feluri.

```
printf("Hello\n");
```




```
#include <stdio.h>

int main()
{
  int N = 3;
  int a = 2;

  if (N >= 10)
   {
      a = 100;
      printf("N >= cu 10\n");
   }
  else if (N < 10 && N >= 0)
   {
      printf("N <= 10\n");
   }
  else {
      printf("N negativ\n");
  }
  printf("a dupa if %d\n", a);
  return 0;
}
```

https://onlinegdb.com/rktVhLrOv

- Declarațiile secvențiale se termină cu ;
- Instrucțiunile de preprocesor nu se termină cu ;
- Declarațiile de selecție, iterație nu se termina cu ;
- Funcțiile nu se termină cu ;
- Comentariile sunt înconjurate de /* */
- Liniile goale sunt ignorate de preprocesor.
- Blocurile de cod sunt marcate de { }

Instrucțiuni C

auto
break
case
char
const
continue
default
do
double
else
enum
extern
float
for
goto
if

Instrucțiunile reprezintă comenzile pe care le dăm compilatorului prin intermediul limbajului C. Împreună cu operatorii (ex: -, +, *, etc.) acestea (in număr de 32) reprezintă întregul conținut al limbajului C. Restul cuvintelor folosite în programele C sunt funcții scrise cu ajutorul acestor instrucțiuni si plasate în biblioteci (ex: stdio.h).

break, **case, etc.** - instrucțiuni de ciclare și selecție

char, **const, etc.** - instrucțiuni de declarare a variabilelor

auto, **extern, etc.** - instrucțiuni de declarare a variabilelor

enum, struct etc. - instrucțiuni de declarare a variabilelor

volatile - evită optimizarea variabilei de către compilator

sizeof - returnează dimensiunea unei variabile

int
long
register
return
short
signed
sizeof
static
struct
switch
typedef
union
unsigned
void
volatile
while

UNDE E PRINTF ? Printf nu este o instrucțiune C. Limbajul C nu conține instrucțiuni IO. printf este parte din biblioteca de sistem, lista de funcții din stdio.h.

stdio.h

```
void      clearerr(FILE *);
char     *ctermid(char *);
char     *cuserid(char *);(LEGACY)
int       fclose(FILE *);
FILE     *fdopen(int, const char *);
int       feof(FILE *);
int       ferror(FILE *);
int       fflush(FILE *);
int       fgetc(FILE *);
int       fgetpos(FILE *, fpos_t *);
char     *fgets(char *, int, FILE *);
int       fileno(FILE *);
void      flockfile(FILE *);
FILE     *fopen(const char *, const char *);
int       fprintf(FILE *, const char *, ...);
int       fputc(int, FILE *);
int       fputs(const char *, FILE *);
size_t    fread(void *, size_t, size_t, FILE *);
FILE     *freopen(const char *, const char *, FILE *);
int       fscanf(FILE *, const char *, ...);
int       fseek(FILE *, long int, int);
int       fseeko(FILE *, off_t, int);
int       fsetpos(FILE *, const fpos_t *);
long int  ftell(FILE *);
off_t     ftello(FILE *);
int       ftrylockfile(FILE *);
void      funlockfile(FILE *);
size_t    fwrite(const void *, size_t, size_t, FILE *);
int       getc(FILE *);
int       getchar(void);
int       getc_unlocked(FILE *);
int       getchar_unlocked(void);
```

```
int       getopt(int, char * const[], const char);
(LEGACY)
char     *gets(char *);
int       getw(FILE *);
int       pclose(FILE *);
void      perror(const char *);
FILE     *popen(const char *, const char *);
int       printf(const char *, ...);
int       putc(int, FILE *);
int       putchar(int);
int       putc_unlocked(int, FILE *);
int       putchar_unlocked(int);
int       puts(const char *);
int       putw(int, FILE *);
int       remove(const char *);
int       rename(const char *, const char *);
void      rewind(FILE *);
int       scanf(const char *, ...);
void      setbuf(FILE *, char *);
int       setvbuf(FILE *, char *, int, size_t);
int       snprintf(char *, size_t, const char *, ...);
int       sprintf(char *, const char *, ...);
int       sscanf(const char *, const char *, int ...);
char     *tempnam(const char *, const char *);
FILE     *tmpfile(void);
char     *tmpnam(char *);
int       ungetc(int, FILE *);
int       vfprintf(FILE *, const char *, va_list);
int       vprintf(const char *, va_list);
int       vsnprintf(char *, size_t, const char *, va_list);
int       vsprintf(char *, const char *, va_list);
```

Standarde C

1978 K&R C

- limbajul C

1990 C89 (ANSI C)

- prototipuri funcționale
- const, volatile, void, long float, long double
- pointeri către funcții fără dereferențiere

1999 C99

- funcții inline
- long long int, boolean, complex
- tablouri cu lungime variabila

2011 C11

- alinierea structurilor de date
- multi-threading
- renunțarea la funcția gets

2018 C18

- update tehnic, fara caracteristici noi

2021 C2X

Variabile

Variabile

Programele operează cu date. Datele sunt de mai multe feluri: numele (întregi, zecimale), caractere (a,b,c… etc).

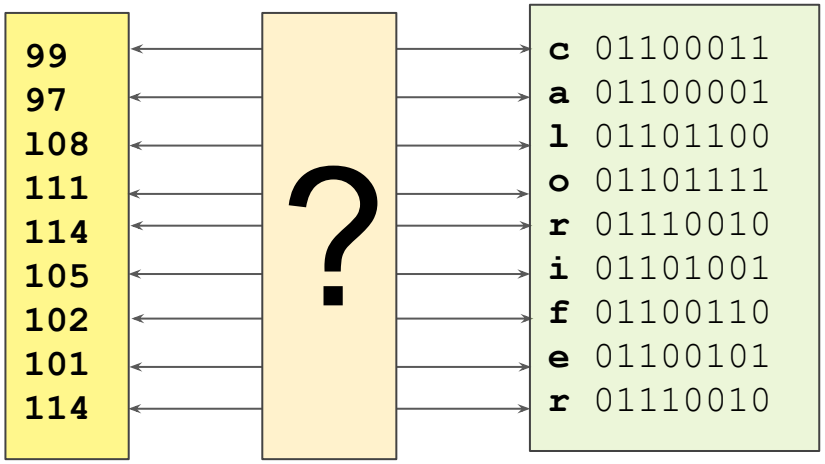
Pentru a opera cu acestea, ele trebuie să fie convertite în format binar și să ajungă în anumite spații de memorie

```
#include <stdio.h>

int main()
{
   int a = 20;
   int b = 30;
   printf("Suma este %i", a + b);
   return 0;
}
```

În memoria computerelor nu se pot stoca decât numere binare.

Pentru a putea stoca texte, e necesară o convenție care să permită stocarea lor sub formă de numere binare.

Compilatorul trebuie să știe însă despre ce e vorba.



Variabilele sunt etichete pentru colecții de adrese de memorie. Acestea se declară în interiorul programelor, adresele corespunzătoare lor sunt alocate în momentul compilării iar apoi programul operează doar cu locații de memorie

Numele variabilelor

Fiecare variabilă trebuie să aibă un nume. Numele acesteia **trebuie să fie unic** (nu pot exista două variabile cu același nume **în același scop**) și trebuie să respecte următoarele reguli:

- Un nume de variabilă poate conține numere și cifre, la care se adaugă semnul _ (underscore)
- Numele NU poate conține spații, virgule sau puncte.
- Un nume de variabilă NU poate începe cu o cifră
- Numele variabilelor nu poate fi aceleași cu cuvintele rezervate ale limbajului (nume de funcții, tipuri de variabile)
- Denumirile variabilelor și restricțiile trebuie să țină seama de faptul că limbajul C este sensibil la capitalizare (case sensitive).

Prin scopul variabilei se înțelege vizibilitatea ei de către alte blocuri de cod. Variabilele sunt:

- **locale** - vizibile doar în blocul de cod (funcție, buclă, bloc decizional) unde au fost definite
- **globale** - vizibile de toate elementele programului.

nume de variabile corecte

```
int suma = 0;
int eu = 12;
char cine_stie_ce[] = "da";
char la_revedere[] = "eh";
int aut112 = 0;
double fprint12 = 1.123;
float frintf_32 = 5.32;
```

nume de variabile incorecte

```
int su ma = 0;
int 12eu = 12;
char cine/stie_ce[] = "da";
char 1_la_revedere[] = "eh";
int aut.112 = 0;
double fprintf-12 = 1.123;
double fprintf = 5.32;
```

Atribuirea variabilelor

```
int x;
x = 2;
```

```
int x = 2;
```

```
int x = 2, y=5;
```

The $20 Million Bug

In Spring 1993, in the Operating System development group at SunSoft, we had a "priority one" bug report come in describing a problem in the asynchronous I/O library. The bug was **holding up the sale of $20 million worth of hardware** to a customer who specifically needed the library functionality, so we were extremely motivated to find it. After some intensive debugging sessions, the problem was finally traced to a statement that read :

```
x==2;
```

The statement as written **compared x to 2**, generated true or false, and discarded the result.

Peter van der Linden - Expert C Programming - deep C secrets

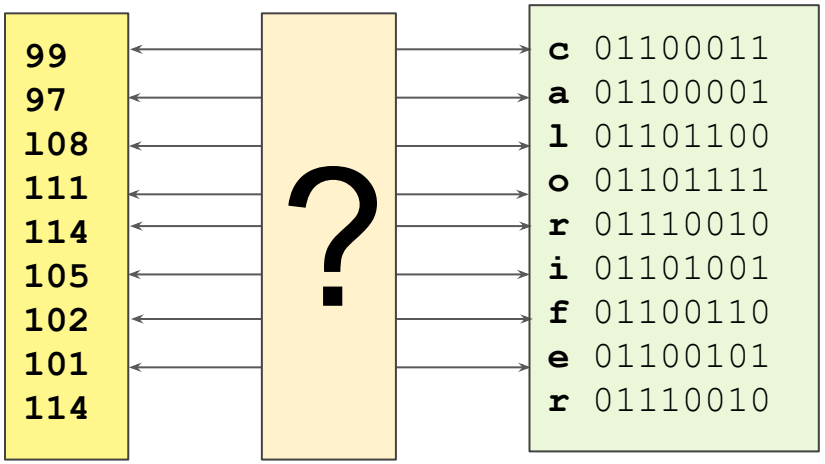
Tipul variabilelor - numere întregi

tip	dimensiune	minimum	maximum
char	1 octet	-127	127
unsigned char	1 octet	0	255
int	4 octeti	-2147483648	2147483648
unsigned int	4 octeti	0	4294967295
short	2 octeti	-32768	32768
unsigned short	2 octeti	0	65535
long	8 octeti	-9223372036854775808	9223372036854775808
unsigned long	8 octeti	0	18446744073709551615
long long	8 octeti	-9223372036854775808	9223372036854775808
unsigned long long	8 octeti	0	18446744073709551615

Tipul char are și altă utilizare

C nu are nici un tip special de date pentru stocarea textelor. Acestea se stochează cu ajutorul tipului **char** (număr întreg pe 8 biți) și a unei convenții cunoscută sub numele **ASCII** (American Standard Code for Information Exchange).

Codul ASCII asociază câte un caracter fiecărui număr întreg care se poate stoca în 8 biți.



Chiar dacă tipul **char** este utilizat pentru șiruri de caractere, acestea pot fi stocate (ineficient) în orice tip de variabila pentru numere întregi.

char_ex.c

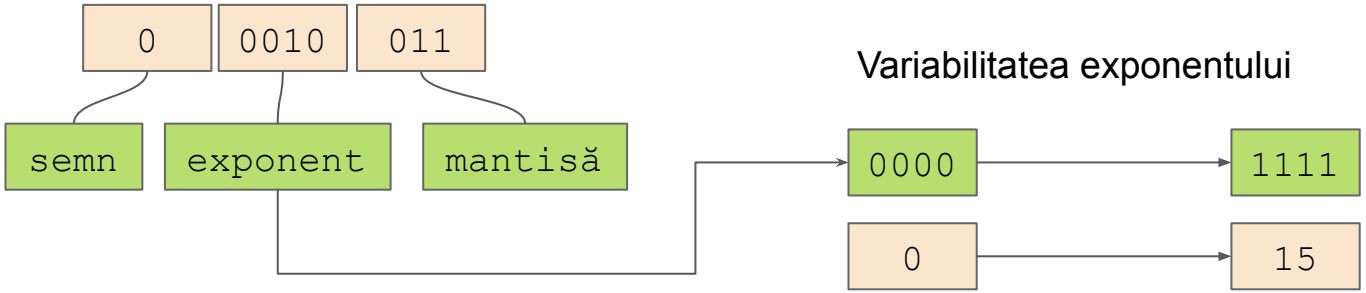
```c
#include <stdio.h>

int main()
{
    char v = 89;
    char f = 'a';
    printf("v intreg: %i\n", v);
    printf("v caracter: %c\n", v);
    printf("f intreg: %i\n", f);
    printf("f caracter: %c\n", f);

    return 0;
}
```

https://onlinegdb.com/SyuykO46Q

81	51	1010001	121	Q
82	52	1010010	122	R
83	53	1010011	123	S
84	54	1010100	124	T
85	55	1010101	125	U
86	56	1010110	126	V
87	57	1010111	127	W
88	58	1011000	130	X
89	59	1011001	131	Y
90	5A	1011010	132	Z
91	5B	1011011	133	[
92	5C	1011100	134	\
93	5D	1011101	135	]
94	5E	1011110	136	^

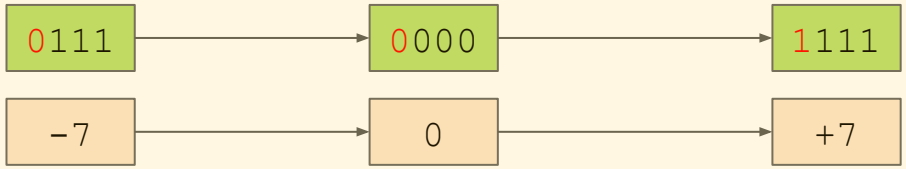
Fragment din tabela ASCII.
https://commons.wikimedia.org/wiki/File:ASCII-Table.svg

Stocarea cu virgulă mobilă - exemplu în 8 biți



Unde sunt exponenții negativi? În lipsa lor nu putem stoca numele subunitare

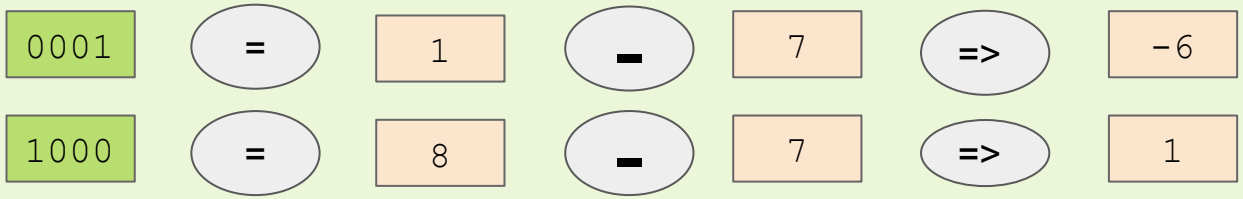
1: folosim unul dintre biții exponentului pentru a stoca semnul



NU

2: folosim "exponent bias" = 2k-1 unde k este numărul de biți din exponent

exponentul are 4 biți => k=4 => exponent bias = 2 * 4 -1 = 7



DA

Exponent bias păstrează același interval de valori dar oferă un mod mai eficient de citire a acestora

https://en.wikipedia.org/wiki/Floating-point_arithmetic

8 biți virgulă mobilă - ce numere putem stoca?

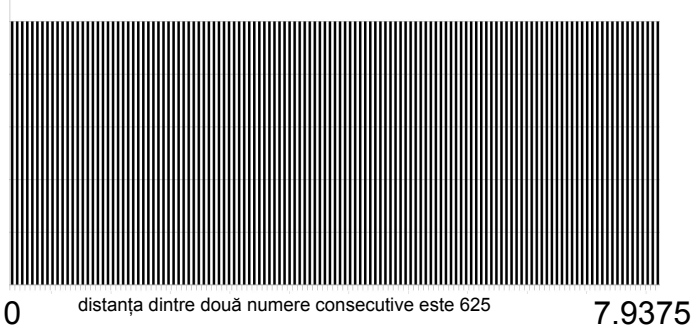
stocare	decimal	binar
00000001	0.125	0.001
00000010	0.250	0.00001
NU	0.31251	0.**0101**000000000000101
00000011	0.375	0.011
00000100	0.5	0.1
NU	0.51	0.**1000**001010001111011
00000101	0.625	0.101
NU	0.62520	0.1010000000001101001

01100000	2048	**1000**00000000
NU	2100	**1000**00110100
01100001	2304	**1001**00000000

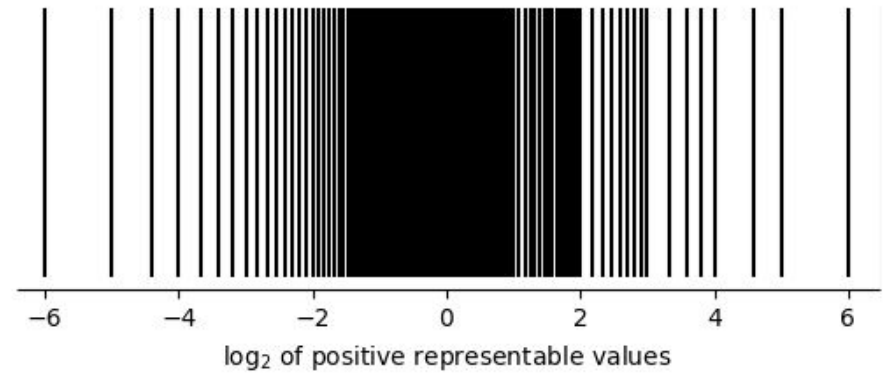
Câteva lucruri interesante despre numerele flotante

Sistemul de stocare a numerelor în virgulă mobilă e extrem de ingenios dar nu poate depăși limitările naturale ale stocării în format binar. Astfel, 8 biți pot stoca 256 de numere diferite și nu mai multe de atât, indiferent de modul în care le reprezentăm.

Densitatea numerelor cu virgulă fixă pozitive stocate in 8 biti.



Densitatea numerelor flotante stocate in 8 biti.



Cu cat numărul de biți în care sunt stocate numerele flotante **crește**, cu atât **crește și precizia acestora** - numerele stocate sunt mai apropiate de numerele introduse. Dar, numerele stocate în acest mod nu sunt niciodată precise până la capăt.

Distanța dintre numerele care pot fi stocate ca float-uri **crește exponențial** cu cât numărul se îndepărtează (spre plus sau minus) de zero.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

Stocarea numerelor în variabile flotante

0.7	semn		exponent		mantisa		total
float	0	1b	01111110	8b	01100110011001100110011	23b	32 biți
double	0	1b	01111111110	11b	0110011001100110011001100110011001100110011001100110	52b	64 biți

float	0.69999998807907104492187 5
double	0.69999999999999999555910790149 9

Valorile stocate în realitate

1.1	semn		exponent		mantisa		total
float	0	1b	0111111	8b	10001100 11001100 11001101	23b	32 biți
double	0	1b	01111111111	11b	0001100110011001100110011001100110011001100110011010	52b	64 biți

float	1.10000002384185791015625
double	1.10000000000000008881784197001

Valorile stocate în realitate

https://www.exploringbinary.com/floating-point-converter/ https://www.binaryconvert.com

Sistemul de stocare al numerelor flotante este un sistem de compresie aproximativa.

Nu se poate stoca orice număr, indiferent de dimensiunea în biți.

Operațiile aritmetice NU vor avea tot timpul rezultatul corect.

Operatori ca =, <, > pot returna rezultate greșite!!

Tipuri de date cu virgulă mobilă disponibile în C

tip	dim	interval	cea mai mică valoare pozitivă	nr. zecimale
float	4 octeti	±3.4E+38	1.2E-38	6
double	8 octeti	±1.7E+308	2.3E-308	15
long double	10 octeti	±1.1E+4932	3.4E-4932	19

printf convertește automat float-ul la double. Evident, cantitatea de date stocate în float nu se modifică dar această conversie ne permite să tratăm floatul ca double pentru a vedea precizia.

```
-------------FLOATS------------
a este: 1.111111164093018
b este: 1111.111083984375000
c este: 11111111.000000000000000
d este: 111111110656.000000000000000
e este: 1111111114326016.000000000000000
-------------DOUBLES------------
m este: 1.111111111111111
n este: 1111.111111111111086
o este: 11111111.111111111938953
p este: 111111111111.111114501953125
q este: 1111111111111111.109375000000000
```

floats_ex.c

```c
#include <stdio.h>

int main() {

  float a = 1.111111111111111111111111;
  float b = 1111.111111111111111111111;
  float c = 11111111.111111111111111111;
  float d = 111111111111.111111111111111;
  float e = 1111111111111111.1111111111111;

   printf ("-------------FLOATS------------\n");
   printf ("a este: %.15f\n", a);
   printf ("b este: %.15f\n", b);
   printf ("c este: %.15f\n", c);
   printf ("d este: %.15f\n", d);
   printf ("e este: %.15f\n", e);

  double m = 1.111111111111111111111111;
  double n = 1111.111111111111111111111;
  double o = 11111111.111111111111111111;
  double p = 111111111111.111111111111111;
  double q = 1111111111111111.1111111111111;

   printf ("-------------DOUBLES------------\n");
   printf ("m este: %.15lf\n", m);
   printf ("n este: %.15lf\n", n);
   printf ("o este: %.15lf\n", o);
   printf ("p este: %.15lf\n", p);
   printf ("q este: %.15lf\n", q);

  return 0;
}
```

https://onlinegdb.com/rk5nMuSuw

Dacă mărimea nu contează

Tipuri de date întregi	
char	1 octet
unsigned char	1 octet
short	2 octeti
unsigned short	2 octeti
int	4 octeti
unsigned int	4 octeti
long	8 octeti
unsigned long	8 octeti
long long	8 octeti
unsigned long long	8 octeti

Caractere — Char

Siruri de caractere — Char[]

C are doar două tipuri de date: numere întregi și flotante.

Numere flotante

tipuri de date flotante	
float	4 octeti
double	8 octeti
long double	10 octeti

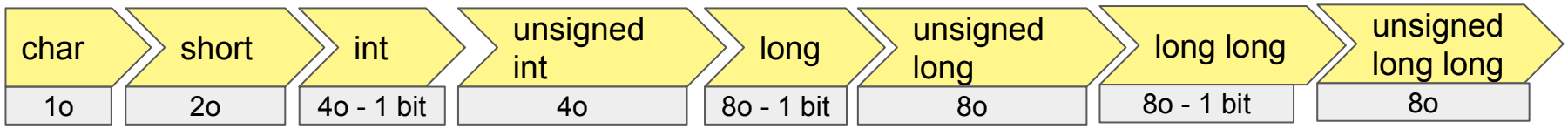
Numere intregi

Toate tipurile de date fundamentale în C au **o corespondență directă în hardware**. Nu există tipuri de bază pentru numere complexe (ca în FORTRAN). Compilatorul nu trebuie să facă eforturi de a stoca tipuri complicate de date.

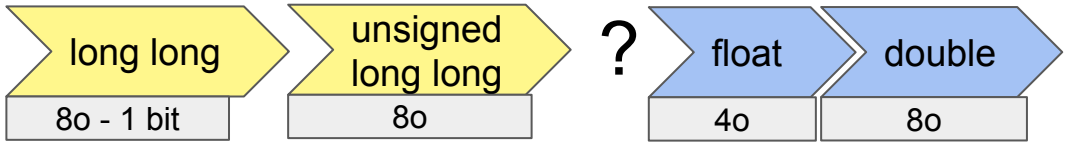
C a început să permită float-uri odată cu apariția suportului hardware.

Conversii de tip automate

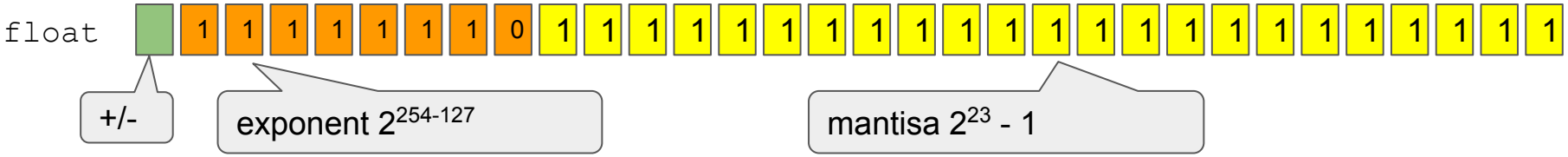
variabilele sunt convertite în tipuri cu spațiu de stocare mai mare sau egal



Compilatoarele permit și conversii de tipul:



Cum putem trece de la 8 octeți la 4 octeți?



Cel mai mare număr întreg care poate fi stocat într-un float (32):

$2^{254-127} \cdot (1 + (2^{23}-1)/2^{23}) = 2^{127} \cdot (2-2^{-23}) = 2^{128}-2^{104}$

=

340282346638528859811704183484516925440

Cel mai mare număr întreg (unsigned long long):

18446744073709551615

modul de stocare a numerelor flotante permite acceptarea unor numere întregi mai mari decât cel mai mare unsigned long long.

conversia unsigned long long -> double este imprecisă

18446744073709546496
18446744073709548544

18446744073709547520

semn: 0
exp: 10000111110
mantisa: 1111111111111111111111111111
1111111111111111111111110

18446744073709548545
18446744073709550591

18446744073709549568

semn: 0
exp: 10000111110
mantisa: 1111111111111111111111111111
1111111111111111111111111

18446744073709550592
18446744073709551615
18446744073709553664

18446744073709551616

semn: 0
exp: 10000111111
mantisa: 0000000000000000000000000000
0000000000000000000000000

18446744073709553665
18446744073709557759

18446744073709555712

semn: 0
exp: 10000111111
mantisa: 0000000000000000000000000000
0000000000000000000000001

Pentru operații precise cu numere mari, se folosesc biblioteci auxiliare. Exemplu, https://gmplib.org/

Conversii de tip explicite

```
#include <stdio.h>

int main() {

  int a = 17, b = 5;
  double c;

  c = a / b;

  printf("Rezultatul este: : %f\n", c );

  c = (double) a / b;

  printf("Rezultatul este : %f\n", c );

  return 0;
}
```

https://onlinegdb.com/BJy1mjDvP

sum este int, count este int. În mod normal, operațiile cu numere întregi produc numere întregi.

Împărțirea a două numere întregi va produce partea întreagă a rezultatului real. 17/5 = 3.4, partea întreagă este 3. Această valoare ajunge în variabila c

c este float, așa că va fi afișat cu zecimale

(double) pus în fața operației determină conversia rezultatului la tipul double. Astfel, în variabila c va ajunge rezultatul corect.

```
Rezultatul este: 3.000000
Rezultatul este:: 3.400000
```

Această operație se numește conversie de tip explicită (type casting).

Conversii de tip explicite

```c
#include <stdio.h>

int main()
{
    float a = 1.1;
    float b = 2.2;

    float c = a + b;

    float d = (int) (a + b);

    float e = (int) a + b;

    printf ("c este: %f\n", c);
    printf ("d este: %f\n", d);
    printf ("e este: %f\n", e);
    return 0;
}
```

https://onlinegdb.com/B12omoruP

Operația adună două numere flotante și pune suma în cel de-al treilea număr flotant. Rezultatul este corect, 3.3.

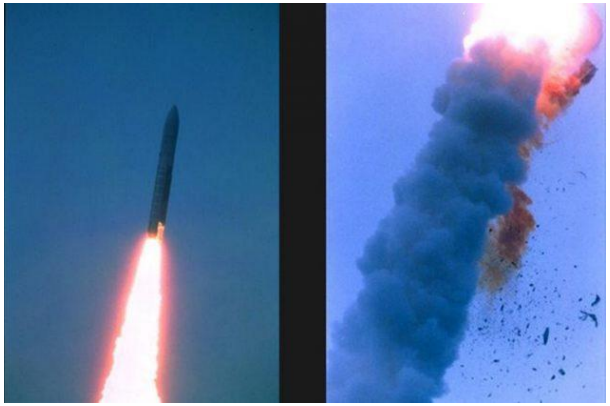
Operatia de cast afectează rezultatul operației a + b (datorită parantezelor). Rezultatul final va fi un număr întreg.

Operația de cast afectează doar variabila a pe care o transformă în număr întreg (1). Rezultatul va fi 1 + 2.2 adică 3.2.

```
c este: 3.300000
d este: 3.000000
e este: 3.200000
```

Conversia implicită de tip

Explozia rachetei Arianne 5 (4 iunie, 1996)



The internal SRI software exception was caused during execution of a data conversion from 64-bit floating point to 16-bit signed integer value. The floating point number which was converted had a value greater than what could be represented by a 16-bit signed integer.*

https://itsfoss.com/a-floating-point-error-that-caused-a-damage-worth-half-a-billion/

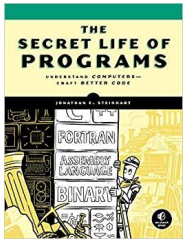
Racheta Patriot, 25 Februarie, 1991

Racheta Patriot nu a reușit să intercepteze o rachetă SCUD lansată de forțele irakiene. Racheta SCUD a lovit o bază americană, omorînd 20 de oameni

Because of the way the Patriot computer performs its calculations and the fact that its registers are only 24 bits long, the conversion of time from an integer to a real number cannot be any more precise than 24 bits. ... Consequently, performing the conversion after the Patriot has been running continuously for extended periods causes the range gate to shift away from the center of the target, making it less likely that the target, in this case a Scud, will be successfully intercepted

http://www-users.math.umn.edu/~arnold//disasters/patriot.html

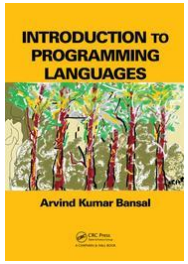
WOULD YOU LIKE TO KNOW MORE?



Jonathan E. Steinhart - **The Secret Life of Programs**
Understand Computers -- Craft better code

No Starch Press, Inc., 2019

ISBN: 978-1593279707



Arvind Kumar Bansal -
Introduction to Programming Languages

Chapman and Hall, 2013

ISBN: 978-0429100338

4 Programming Paradigms In 40 Minutes

O prezentare a principalelor paradigme de programare, cu exemple foarte simple de cod (utilă pentru cei care nu au cunoștințe de programare).

https://www.youtube.com/watch?v=cgVVZMfLjEI

Programming Across Paradigms

O prezentare a principalelor paradigme de programare, cu exemple de cod (utilă pentru cei care au cunoștințe de programare). 49.07 min.

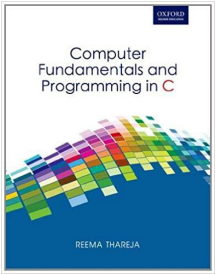
https://www.youtube.com/watch?v=Pg3UeB-5FdA

Building an 8-bit breadboard computer!

Pentru cei interesați de arhitectura computerelor - un playlist care detaliază construcția unui computer pe 8 biti pe o placă de tip breadboard cu fire și leduri. 44 de clipuri a câte 30 min fiecare, în medie.

https://www.youtube.com/playlist?list=PLowKtXNTBypGqImE405J2565dvjafglHU

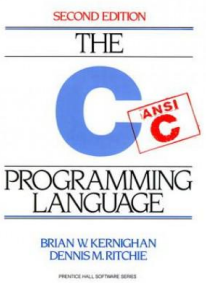
WOULD YOU LIKE TO KNOW MORE?



Pradip Dey, Manas Ghosh - **Computer Fundamentals and Programming in C**

Oxford University Press, 2013

ISBN-13: 978-0198084563



Brian Kernighan, Dennis Ritchie - **C Programming Language**, editia 2

Prentice Hall, 1988

ISBN: 978-0131103627

Uncle Bob Martin - the future of programming

Robert Martin ("unchiul Bob") este un teoretician și practician al principiilor de dezvoltare software, fiind inițiatorul principiilor SOLID și a sistemului AGILE de management de proiect.

https://www.youtube.com/watch?v=ecIWPzGEbFc

History and Spirit of C - Olve Maudal

O prezentare a istoriei dezvoltării programării, ajungând la apariția limbajului C

https://www.youtube.com/watch?v=xGVRF-Y--hI

Brian Kernighan: UNIX, C, AWK, AMPL, and Go Programming | Lex Fridman Podcast

Interviu cu Brian Kernighan - coautor al primului manual de C, participant la proiectarea sistemului UNIX, etc.
https://www.youtube.com/watch?v=O9upVbGSBFo

Oral History of Ken Thompson - Computer History Museum

UNIX, B, Go programming language, UTF-8, ED, expresii regulate

https://www.youtube.com/watch?v=xGVRF-Y--hI